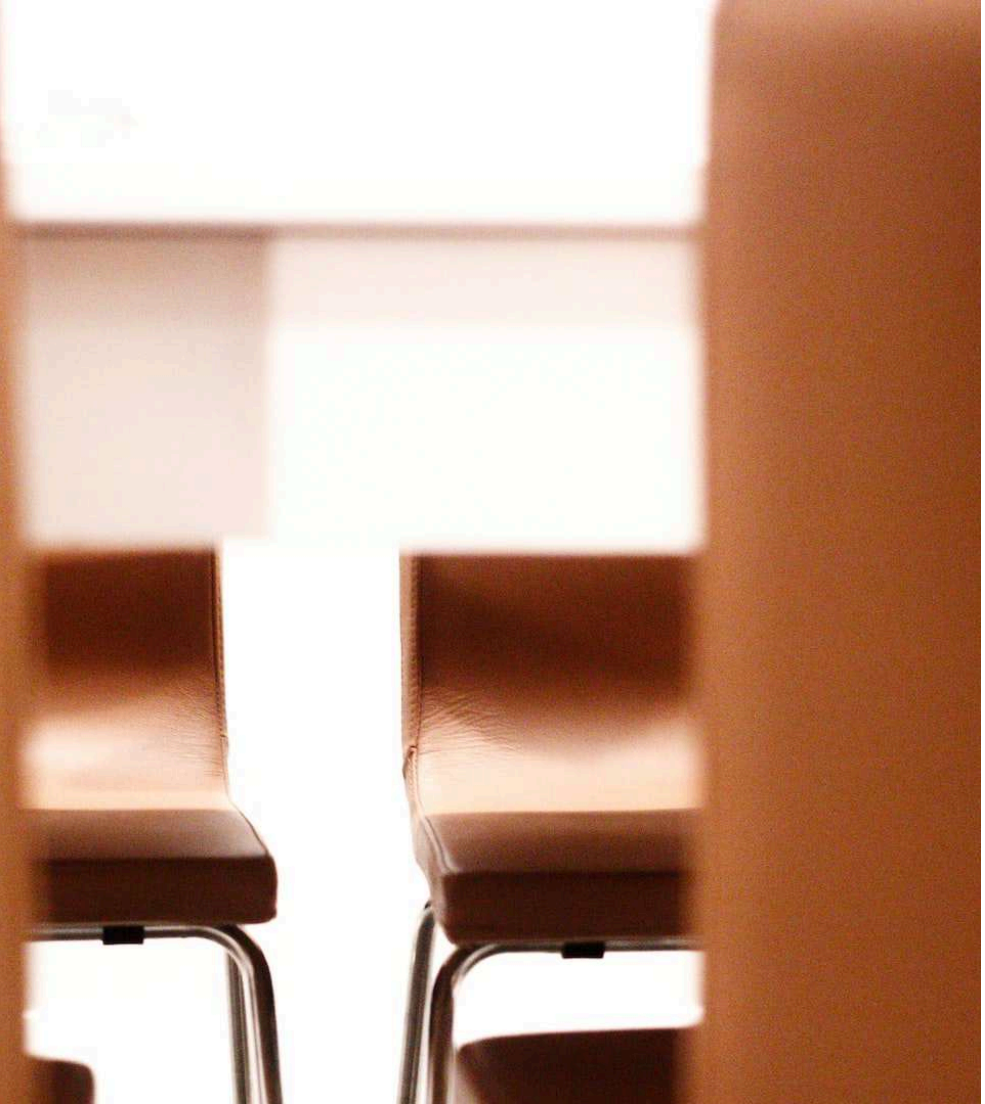
Designing IT systems

Architecture, Practices and Patterns



Training

- Training plan and objectives
- Current responsibilities
- Questions, discussions, needs
- Flexible training program



Mateusz Kulesza

- Senior Software Developer
- Enterprise Architect
- Solution Architect, Project Manager
- Consultant and Trainer

What is software architecture

— and what is it not?



Every system has its architecture

Some architectures are obvious and visible; many others are not…

- The descriptive architecture of a system ultimately resides in its executable code.
- Before building the system, its normative architecture must be clearly defined.
- A system’s architecture can be represented by models abstractly linked to the code (existing or yet to be written).

Architecture is a description or a recipe

Not every architecture is a good one



Architecture vs. Design

Architecture is a design, but not every design is architecture.

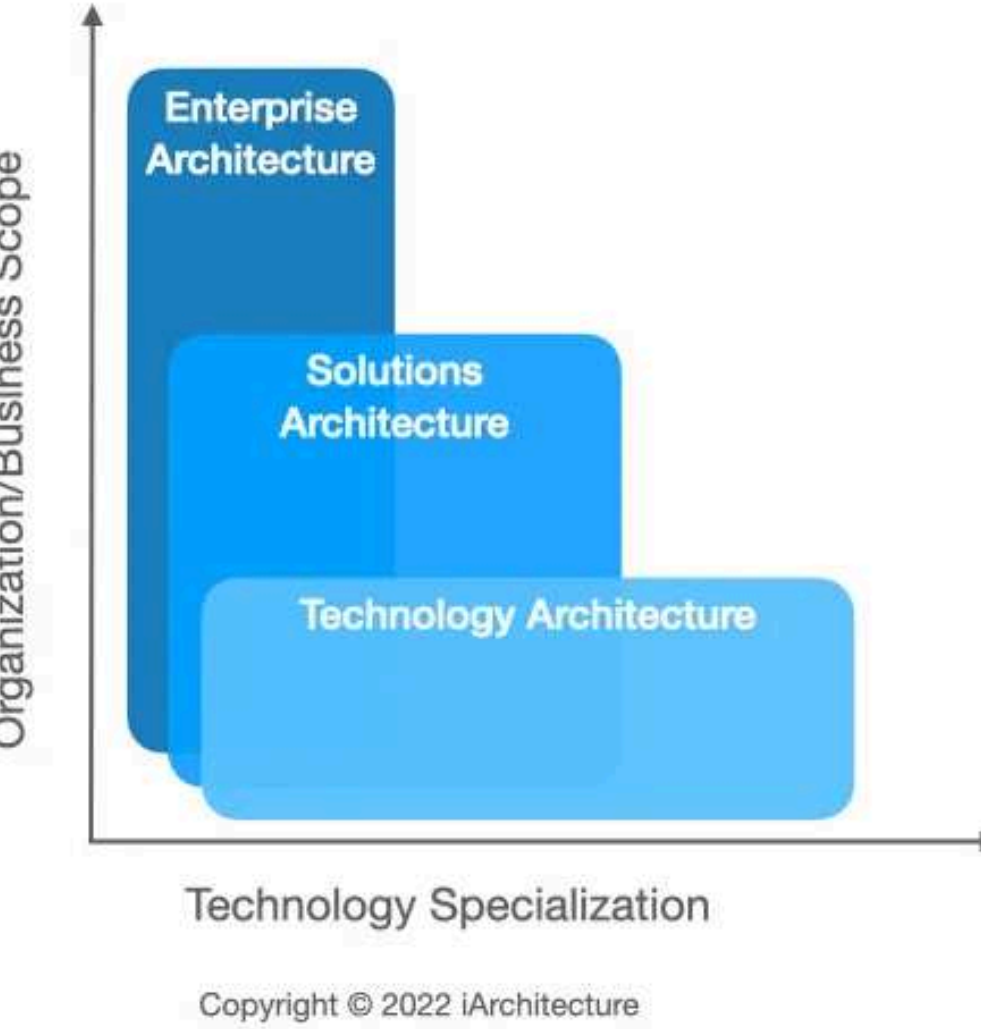
Many design decisions remain open in the architecture.

Their resolution lies with designers in later phases of work, or even with the people responsible for implementation.

Architecture

is not a place, but a journey





IT System Architecture vs. Enterprise Architecture

Enterprise Architecture

In simple terms, it acts as a catalyst for communication between the business, technology, and project areas of the organization. It provides a shared platform for understanding and supports them in daily operations.

IT Architecture

Organizes all IT resources of the organization. It is important to consider all architectural layers during its creation.

Conway’s Law

"Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization’s communication structure."

Architecture shapes the organization’s structure—or **vice versa**.

Stakeholders and roles in the architectural process

Project stakeholders are people who influence the project or are influenced by it.

- Who will be affected (positively and negatively) by your project?
- Who will be interested in your project?
- Who has authority over your project?
- Who wants your project to fail?
- Who wants your project to succeed?

They must be included in the planning and implementation of the architecture.

A good practice is a communication plan involving their continuous engagement.

Tools:

Stakeholder management, stakeholder map, power-interest grid, stakeholder onion, SWOT

Example stakeholders

- Project manager
- Project team members
- Project portfolio manager
- Program manager
- Project sponsor
- Executive management
- Other internal teams
- Customers
- Contractors
- Subcontractors
- Users
- Investors
- Suppliers



... any others?



Measuring architecture

How to measure architecture?

What is the measure of architecture?

- Number of components?
- Number of lines of code?
- Man-hours?

Added value of the modeled IT system

Level 1:

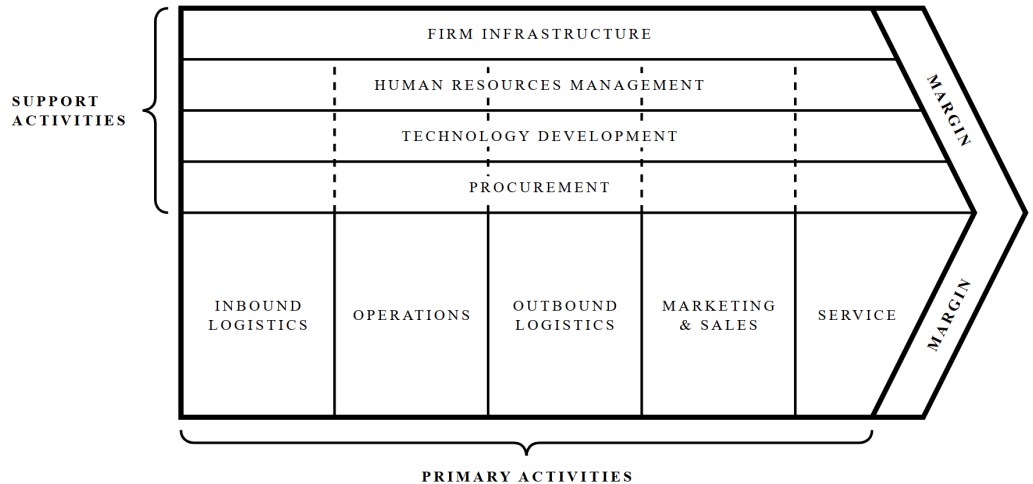
IT as a **cost center**

Level 2:

IT as **part of the company's strategy**

Level 3:

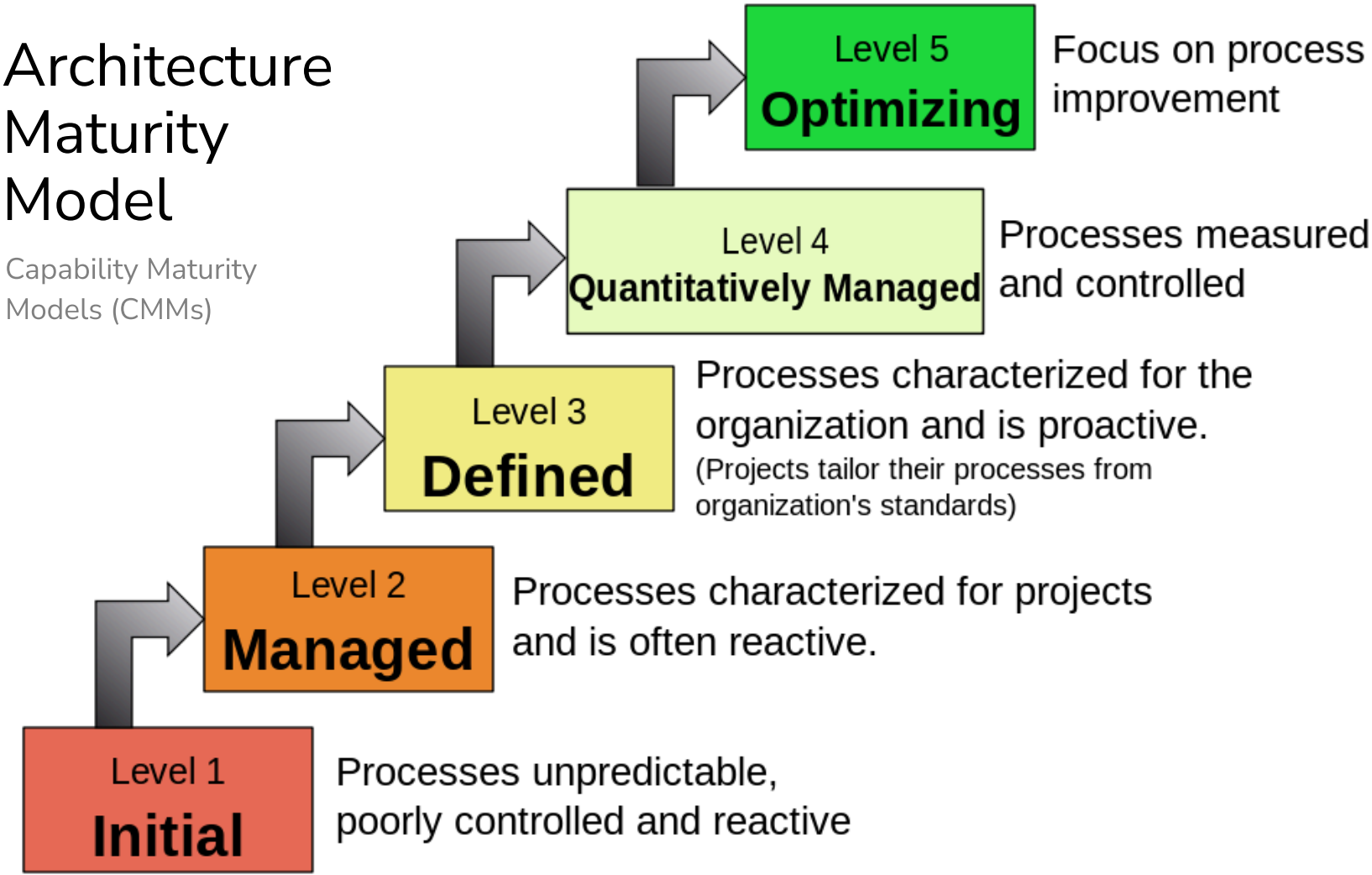
IT as a **catalyst for strategy**



Porter`s Value Chain

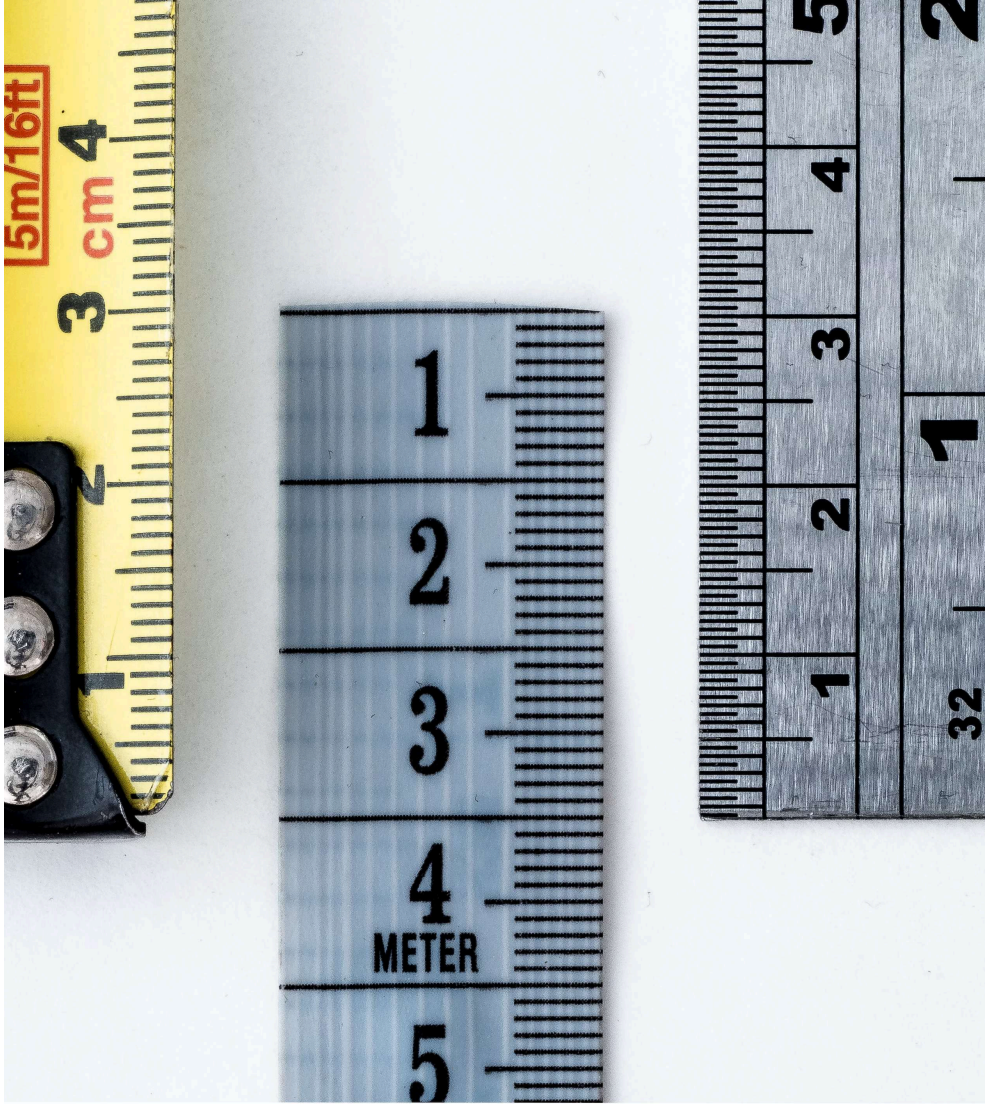
Architecture Maturity Model

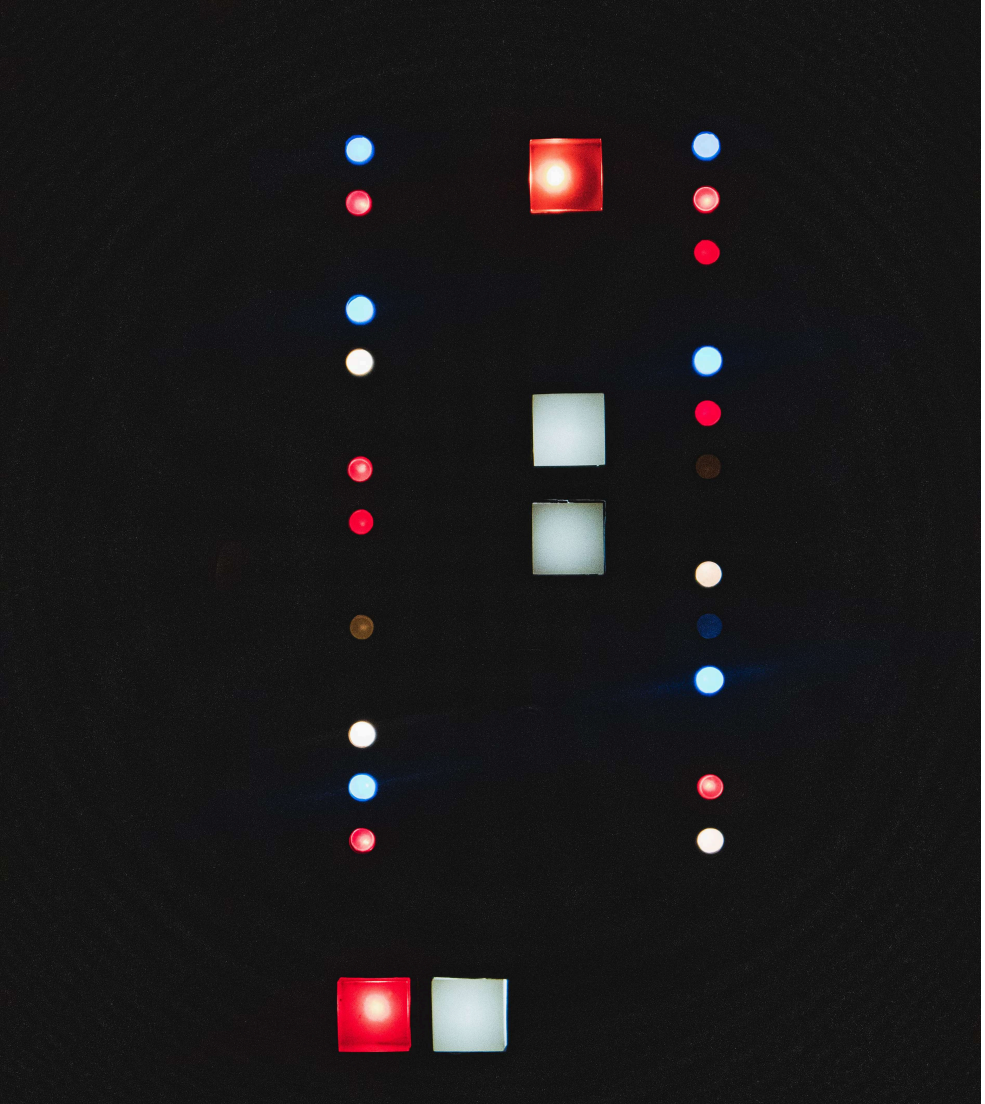
Capability Maturity Models (CMMs)



Architectural Metrics

- Architecture must support the company’s strategies
- Goals and metrics (KPIs, OKRs) of the strategy
- Implementation constraints
- Impact on organizational structure
- Estimating costs and timelines
- Enabling gradual system development





"What will happen
if I press this button?"

Quality attributes for:

- User — who uses it?
- Client — who pays?
- Manager — who manages?
- Developer — who builds?
- Architect — who plans?
- Competitor — who ???

"And what should actually happen?"

Quality Attributes

Non-functional

- Availability
- Ease of Deployment
- Efficiency
- Integrability
- Modifiability
- Performance
- Safety
- Security
- Testability
- Usability

Functional

- Required functionality for the user

A few questions

To think about

- Do you know other definitions of software architecture?
- How does architecture provide a foundation for analysis?
- What about decision-making? What types of decisions can be made thanks to architecture?
- What is the role of architecture in reducing project risks?
- Do you know an example of a software architecture?

Architecture Description

Roles of Architecture:

- Abstraction of the "real" system
- Interpretation of the "logical" model

Architecture is an abstraction

Because architecture consists of structures, and structures include elements and relationships, it follows that architecture describes software elements and how they are connected.

This means that some information about elements not useful for analyzing the system is intentionally omitted. Architecture is primarily an abstraction of the system, including selected details while omitting others.

Architecture includes structure and its behavior

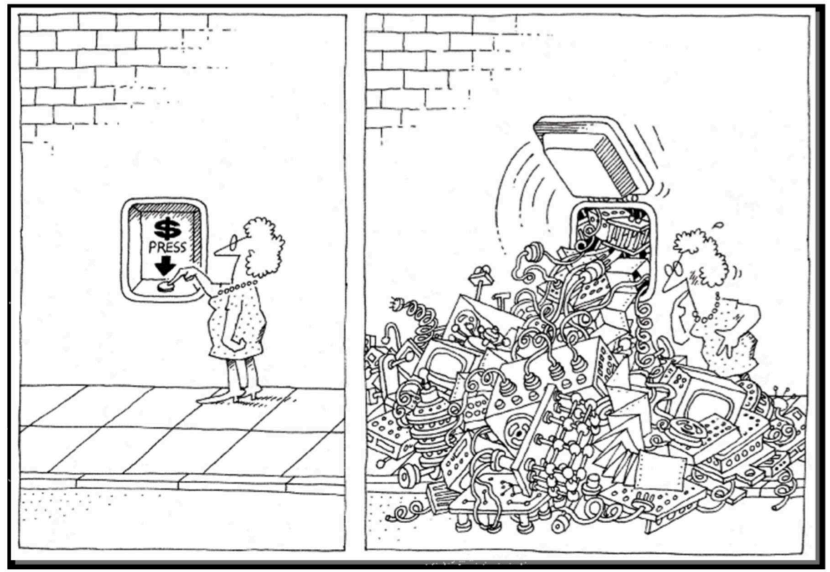
Architecture answers questions like:

- What are the main executing components and how do they communicate?
- What are the main shared storages and data types?
- Which parts of the system are replicated? Where and how?
- How does data move in the system? What data? In which direction?
- Which parts of the system can operate concurrently?
- Can the system’s structure change during execution? How?

And many more…

Architecture

- Should be evaluated for conveying important **quality attributes of the system**
- Should allow **incremental implementation**
- Defines a **set of constraints** on later implementation
- Is the carrier of the most fundamental and **hardest-to-change** design decisions
- Should include well-defined modules, whose functional responsibility is based on principles of **information hiding and task separation**



Architecture as a structure map

Software architecture examines and establishes the structure of software or a system.

Structure includes software elements, their properties, and relationships.

Architecture involves making the most important design decisions about the system.

The motivation for modeling lies in the difficulty humans have scaling the understanding of large amounts of code.

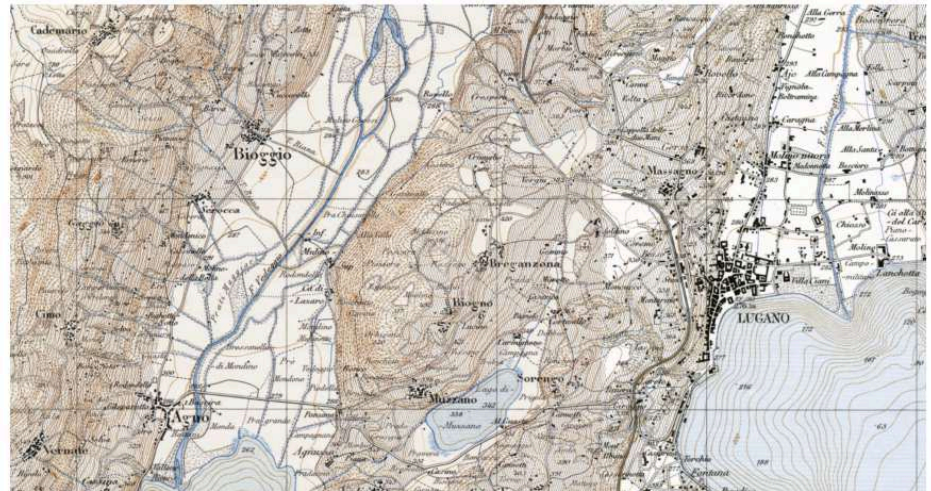
Developers live in a forest of thousands of lines of code.
Sometimes they get lost and need a map.





The map is not the territory

Architecture is not the system but its abstraction



Architecture describes many structures

- Decomposition structure
- Usage structure
- Layered structure
- Domain model
- Class structure (generalizations)
- Data and relationships model
- Communication model
- Services structure
- Concurrency structure
- Deployment structure
- Implementation structure
- Work allocation structure

...and many others

Architectural structures are system abstractions

They hide irrelevant information and divide it by stakeholder goals.

Views

Architecture should be documented using views.

Architectural View:

- Is a representation of one or more architectural structures.
- Facilitates communication between stakeholders.

Model, View, and Perspective

No single modeling approach can capture the full complexity of software architecture.

Model

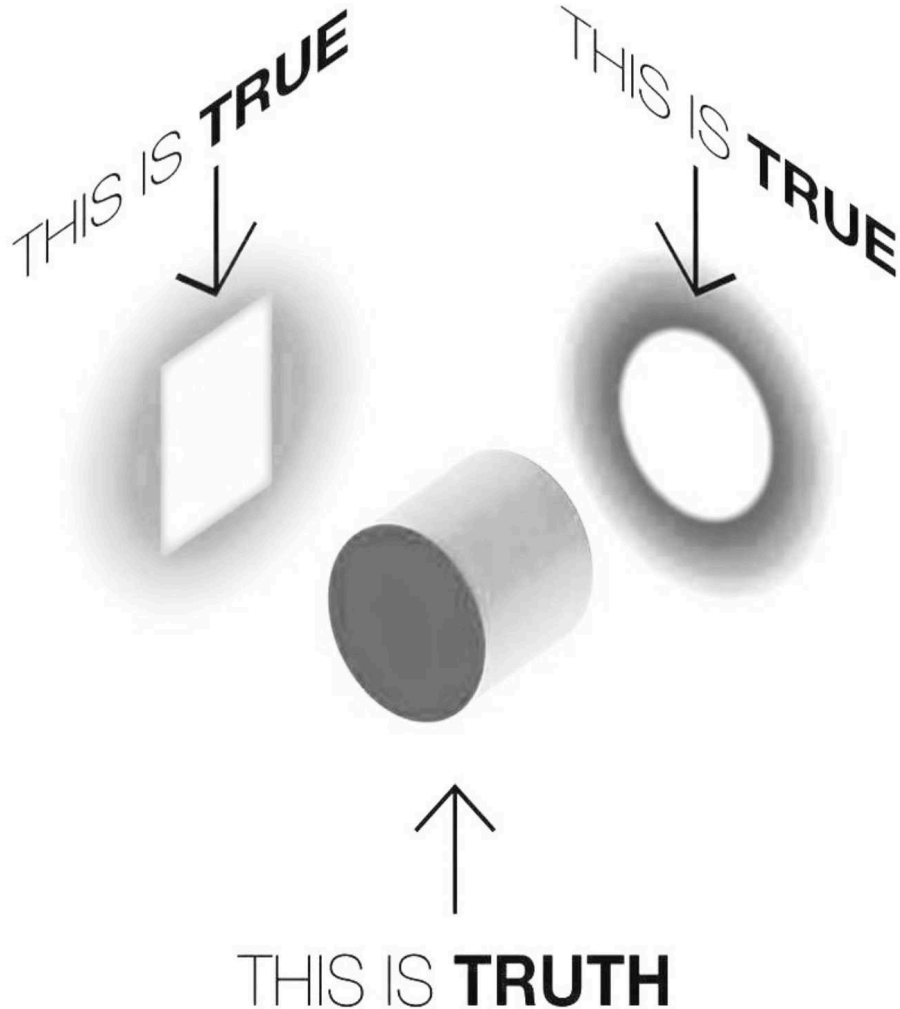
- A complex abstraction of part of the system.

View

- Shows the model from a specific perspective.

Perspective

- Views from the stakeholder's point of view.



Which Views

- System context
- Functional
- Logical
- Physical
- Deployment
- Development
- Informational
- Process
- Competitor
- Operational
- Security
- Performance and scalability
- Availability and reliability
- Regulatory
- Marketing
- Business impact

A view is a representation of the model’s structure.

Architects design models and then document the views of those models.

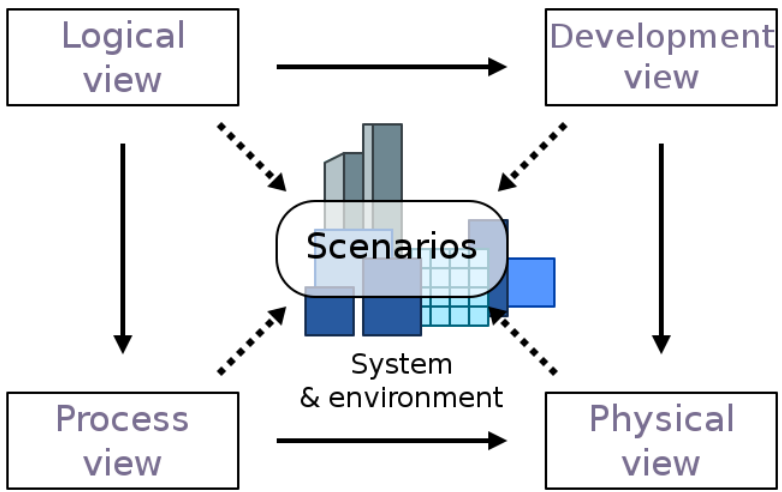
4+1 Model

It consists of four different views plus one called Scenarios.

Each view allows for analysis of different system aspects and is intended for different stakeholders.

Views:

- Logical
- Process
- Deployment
- Physical
- Scenarios



* The “4+1” View Model of Software Architecture. Philippe Kruchten. 1995.

4+1

Logical view

- Intended for end-users. Diagrams in this view focus on presenting system functionality using, e.g., class diagrams or entity-relationship diagrams.

Process view

- Relates to software elements at runtime that can be stopped or started.

Development view

- Diagrams map functional requirements to specific modules, layers, or subsystems.

Physical view

- Shows how software elements are assigned to hardware elements.

Scenarios

- A set of use cases.

4+1 — Scenarios

A set of use cases, or scenarios, forming the fifth view.

Scenarios describe sequences of interactions between objects and between processes.

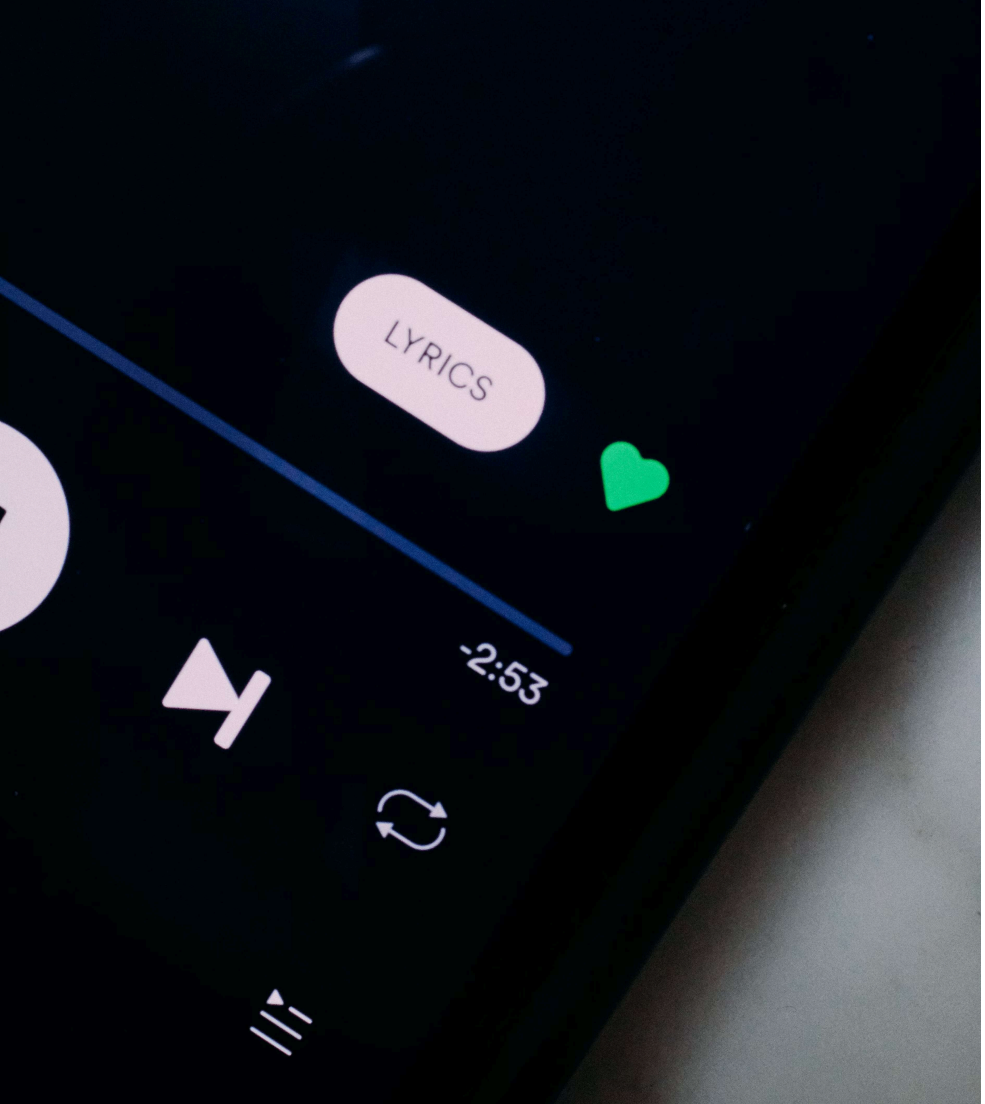
They help identify architectural elements and illustrate and validate the architectural design.

They are also a starting point for architectural prototype testing. This view is also called the use-case view.

Ideas

- Come up with an example application.
- List example use scenarios.





Music Player

Example scenarios

- Search for interesting tracks
- Play a track sample
- Pay to hear the full track
- Download purchased track to device
- Play the full track
- Play sequentially in a playlist
- Play in random order
- Share tracks with friends
- Suggest related tracks
- Display album cover art
- Show device battery status

Selected Notations and Patterns

Modeling

A domain model represents the world and the problems we are trying to solve, and then we create a design model, which is the solution.

- What languages, notations, or metamodels can we use to represent this information as accurately as possible?
- Capture more or fewer details?
- What is the purpose of the model?
- Modeling for the sake of modeling gathers a mass of information that may turn out to be useless

You can draw beautiful diagrams, admire them ...

And only then ask yourself – why even do it?

We invest in creating a model only when there is a concrete reason to do so.

First the Question, Then the Model

- Different models have different purposes
- Before building a model, find out what questions it is supposed to answer

Model Quality

- **Unambiguity** – a model is ambiguous if it leads to more than one interpretation
- **Completeness** – models can be incomplete: different people will fill the gaps in different ways.
- **Accuracy** – a model is accurate if it is correct, consistent with facts, or deviates within acceptable bounds.
- **Precision** – a model is precise if it is very detailed or clearly constrained.

Modeling = Learning

A domain is a set of decisions over which you have no control

- The modeling process is nonlinear – Analysis and Synthesis cannot wait for each other
- Architecturally significant requirements are discovered in the domain as the design progresses
- After creating a solution by writing software, the domain model can be used for acceptance testing.
- Users and clients will evaluate it and see whether it is good enough for them and meets their requirements.
- Domain knowledge should be like a contract specifying all the goals we need to achieve.

CASE Tools

CASE tools used for modeling are often tied to a specific software development methodology.

CASE components include, for example:

- diagram editors of a chosen notation
- model validation tools
- tools for using data repositories
- tools for creating reports and other system documentation
- code generators

Popular tools:

- Sparx Enterprise Architect
- Visual Paradigm
- Star UML
- Argo UML
- IBM Rational Rhapsody
- Microsoft Visio

https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools

Popular Methodologies and Notations

Notations

- C4
- Archimate
- UML
- ERD
- BPMN

Methodologies

- Domain Driven Design
- TOGAF
- Zachman Framework
- Scaled Agile Framework
- others...

C4 Model

Simon Brown created the C4 model* for visualizing software architectures.

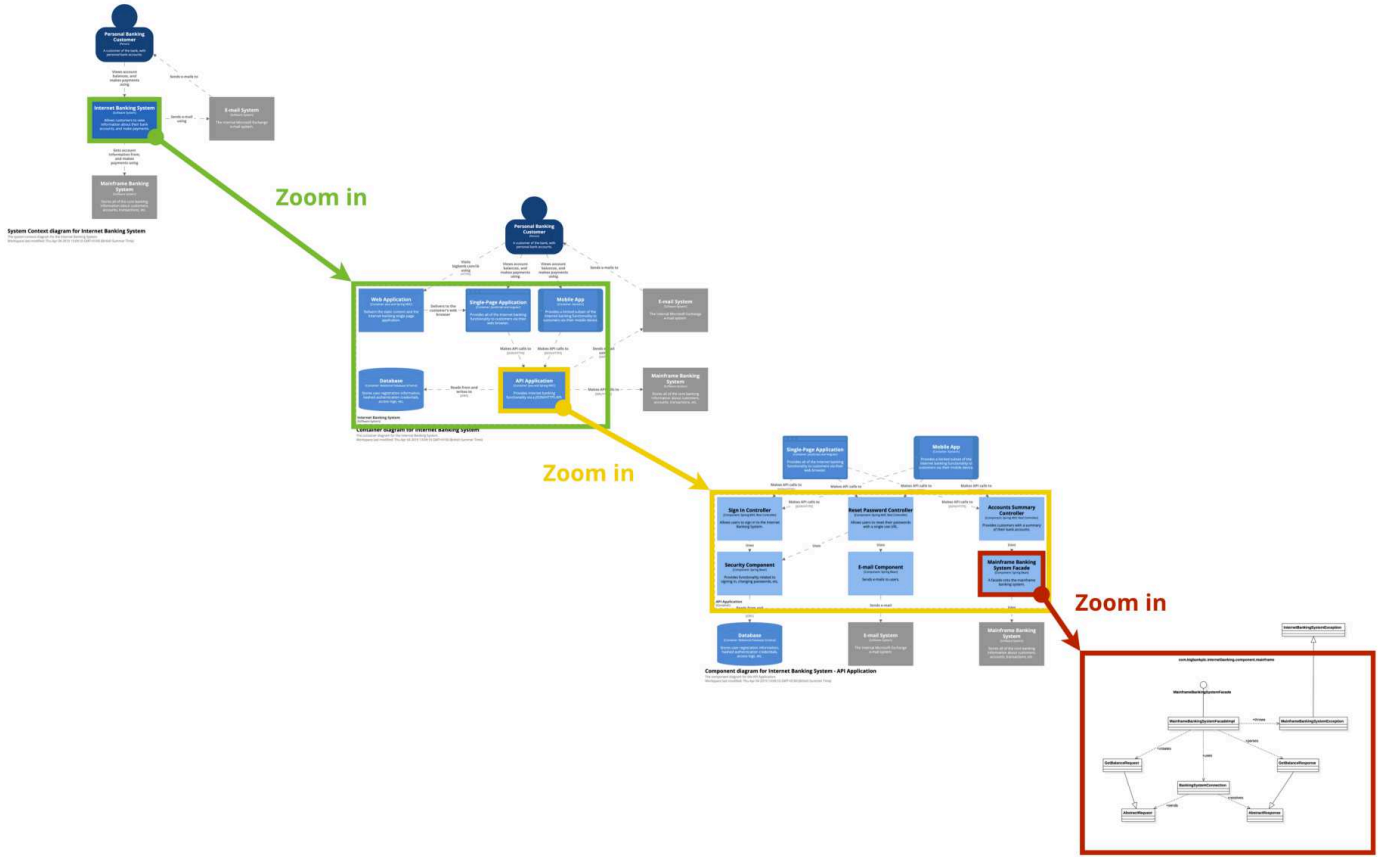
Its name comes from acronyms of various abstraction structures that it allows to diagram:

- Context
- Container
- Component
- Code

It is based on existing modeling techniques, such as:

- Unified Modeling Language (UML)
- Entity Relationship Diagrams (ERD)

for a more detailed breakdown of architectural elements.



Level 1

Context

Level 2

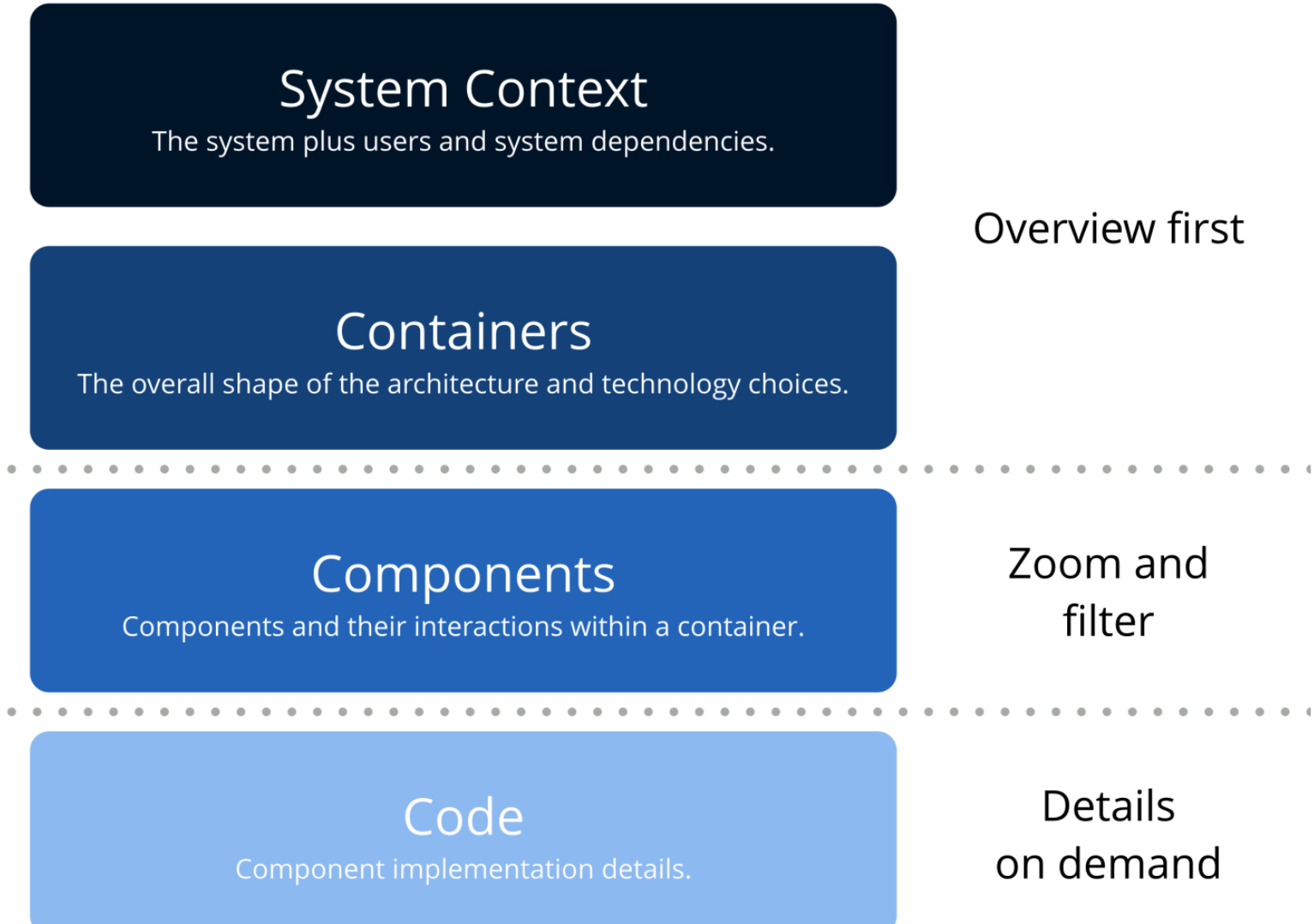
Containers

Level 3

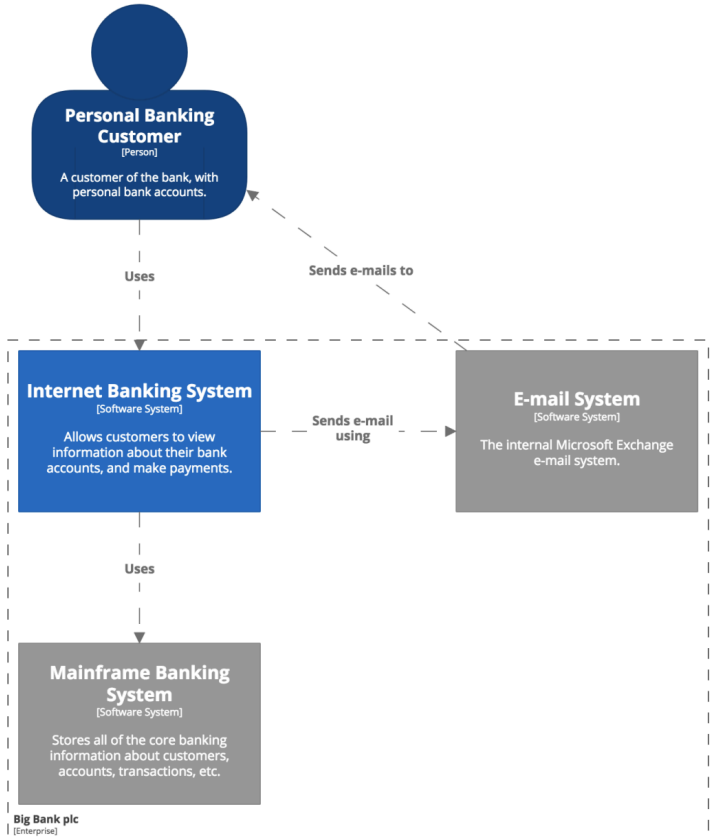
Components

Level 4

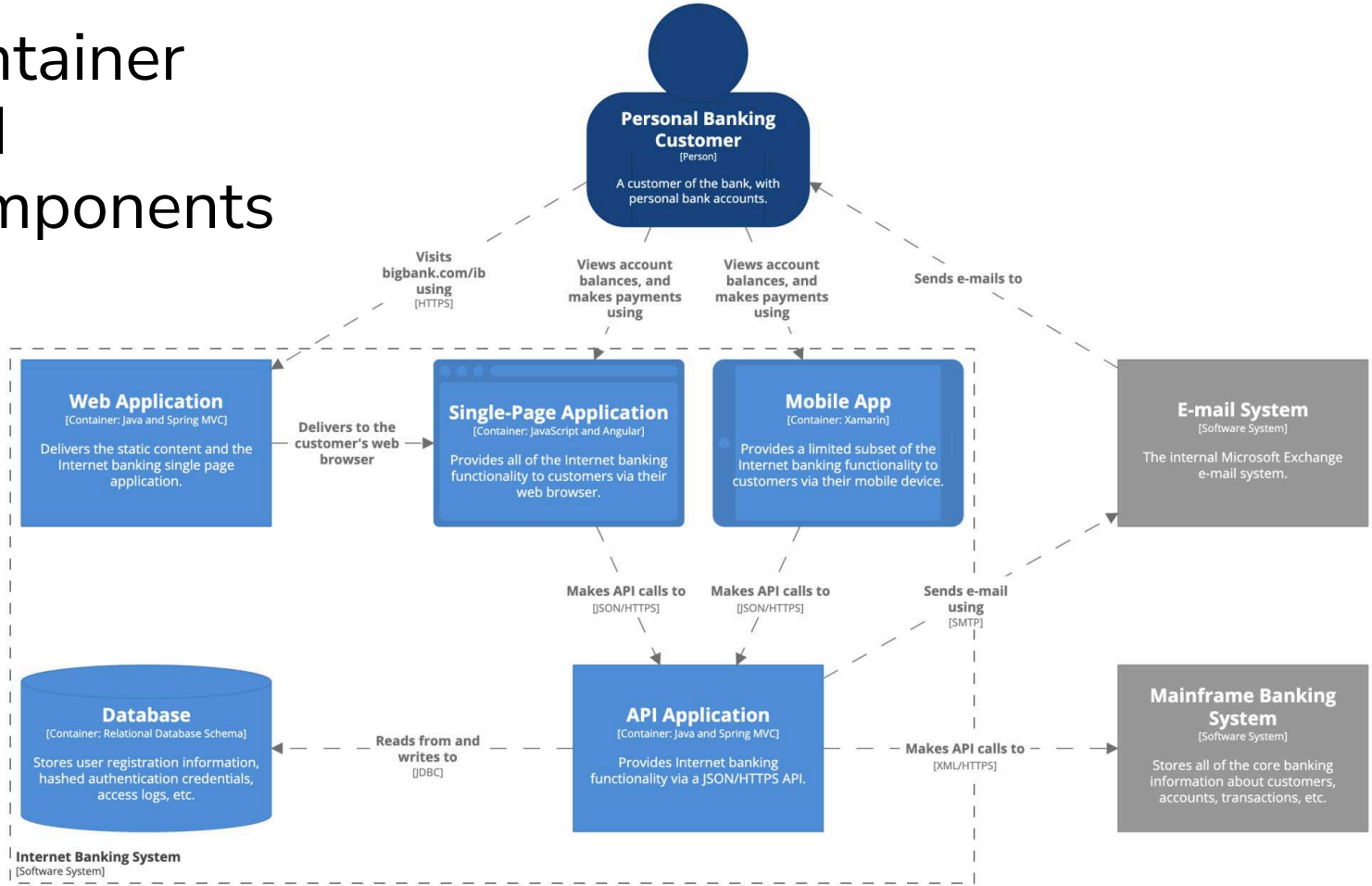
Code

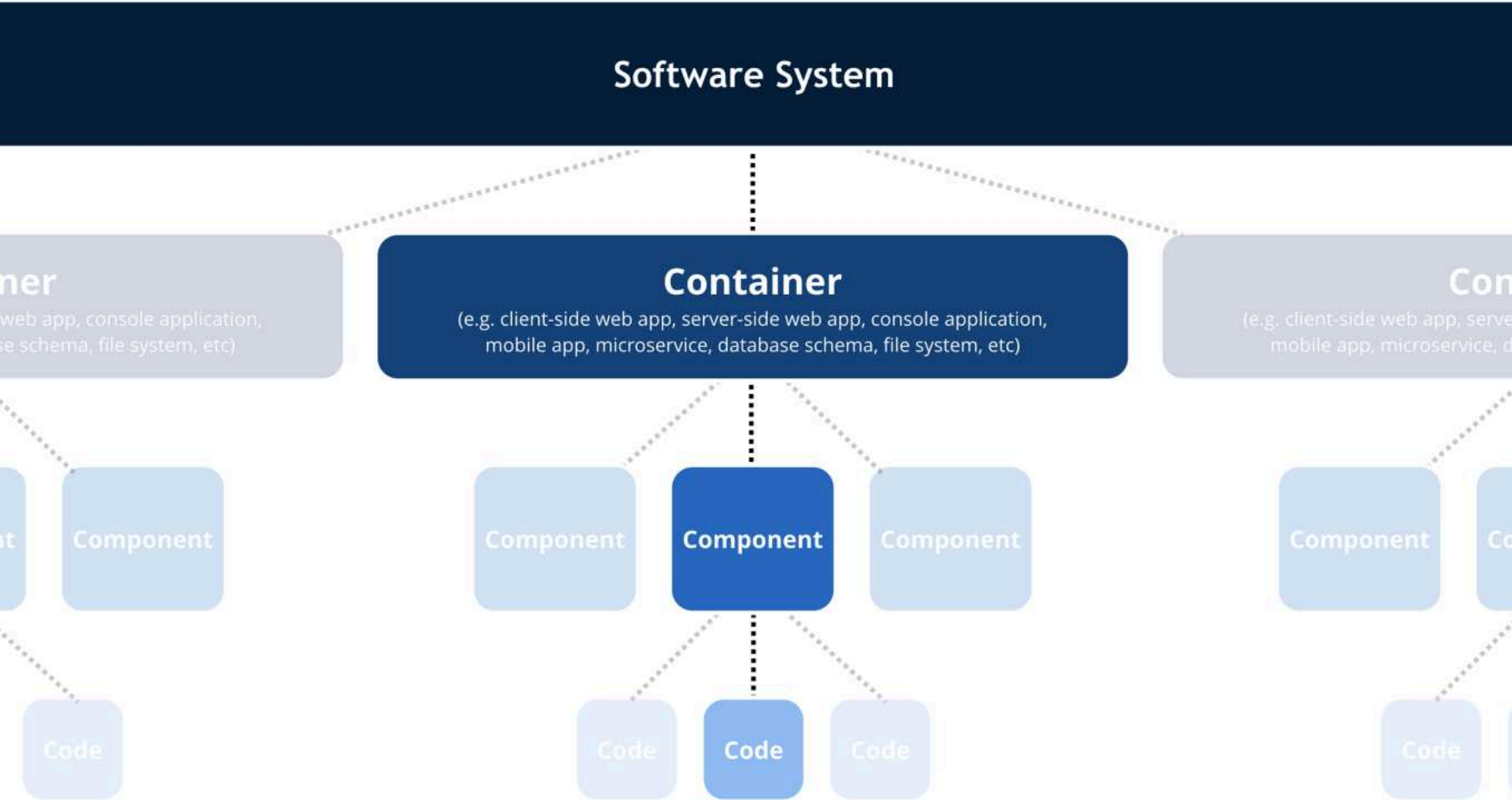

System Context
The system plus users and system dependencies.
Containers
The overall shape of the architecture and technology choices.
Components
Components and their interactions within a container.
Code
Component implementation details.
Overview first
Zoom and filter
Details on demand

Internet Banking System Context



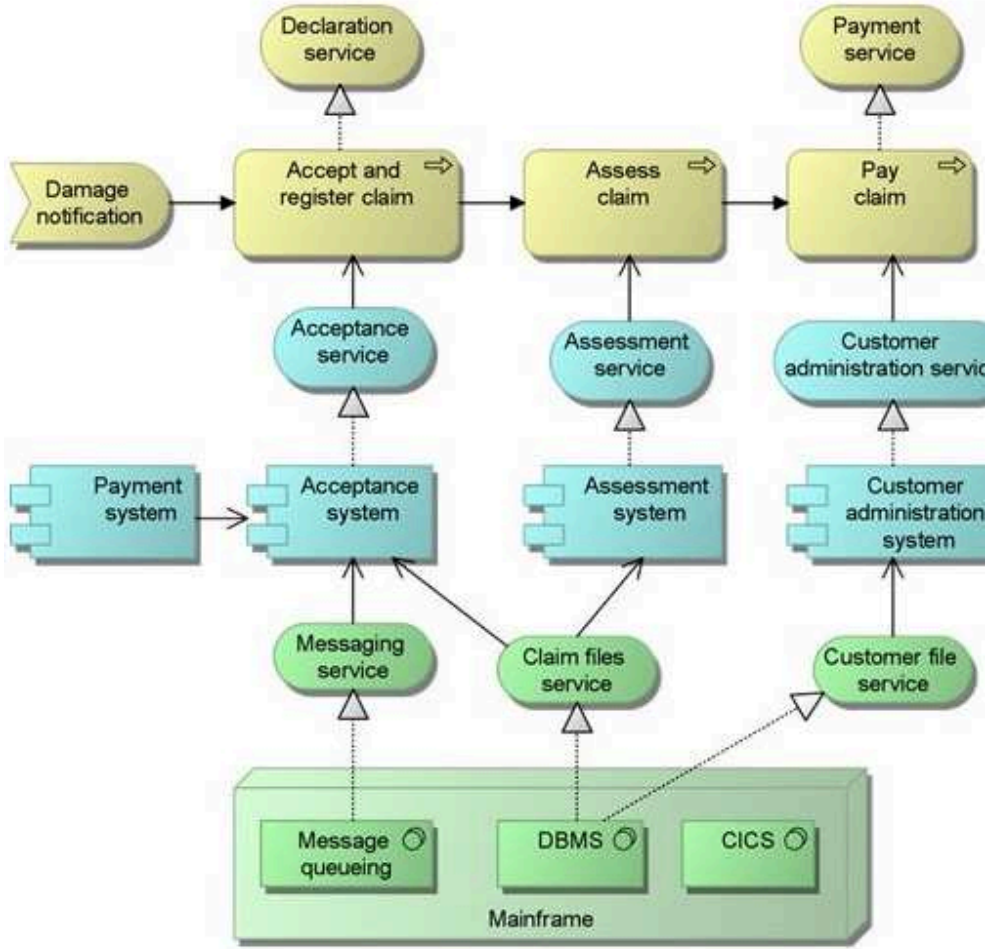
Container and Components




Software System
Container
(e.g. client-side web app, server-side web app, console application, mobile app, microservice, database schema, file system, etc)
Component
Component
Component
Component
Component
Code
Code
Code
Code

Archimate

is an open and independent enterprise architecture modeling language that unambiguously supports the description, analysis, and visualization of architecture within and between business domains



ArchiMate differs from other languages such as:

- Unified Modeling Language (UML)
- Business Process Modeling and Notation (BPMN)

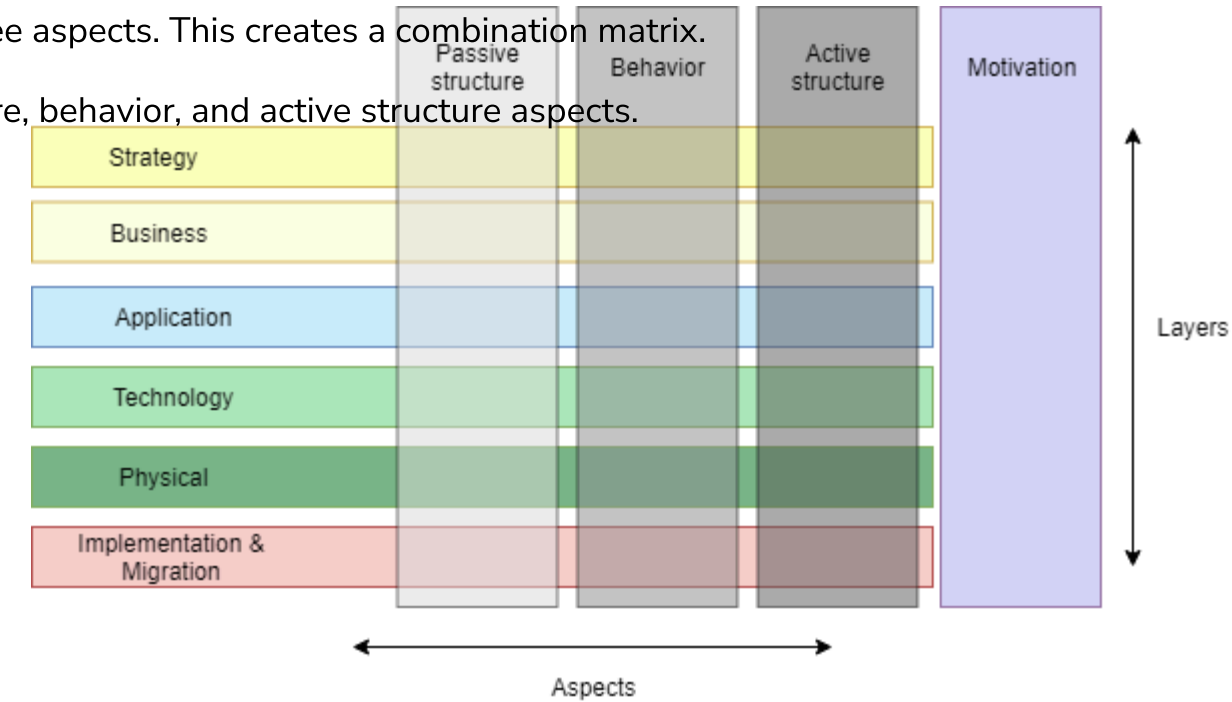
in the scope of enterprise modeling

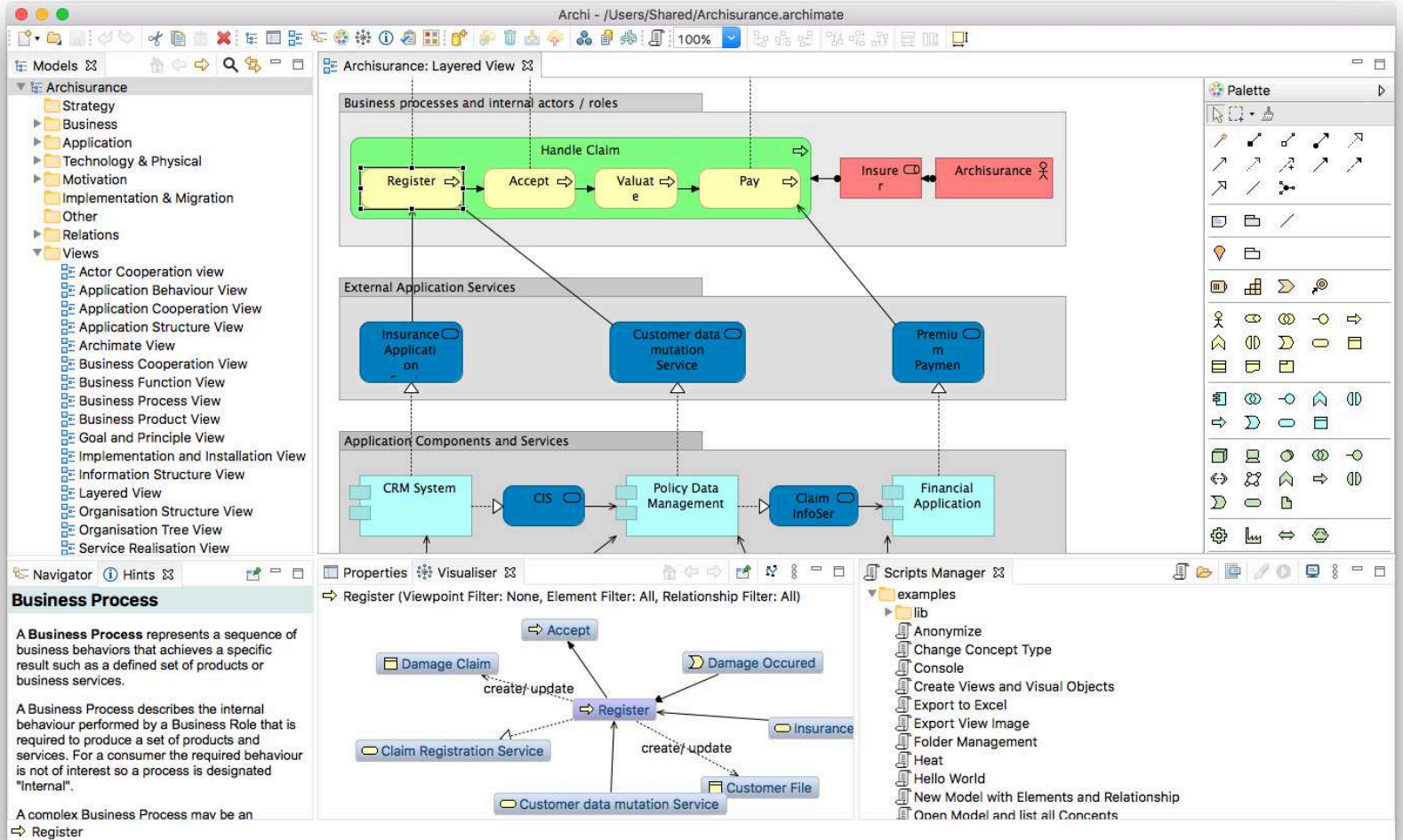
Archimate Framework

The main concepts and elements of the ArchiMate language are presented as the ArchiMate Core Framework.

It consists of three layers and three aspects. This creates a combination matrix.

Each layer has its passive structure, behavior, and active structure aspects.




Archi - /Users/Shared/Archisurance.archimate
Models
Archisurance
Strategy
Business
Application
Technology & Physical
Motivation
Implementation & Migration
Other
Relations
Views
Actor Cooperation view
Application Behaviour View
Application Cooperation View
Application Structure View
Archimate View
Business Cooperation View
Business Function View
Business Process View
Business Product View
Goal and Principle View
Implementation and Installation View
Information Structure View
Layered View
Organisation Structure View
Organisation Tree View
Service Realisation View
Archisurance: Layered View
Business processes and internal actors / roles
Handle Claim
Register
Accept
Valuat e
Pay
Insure r
Archisurance
External Application Services
Insurance Applicati on
Customer data mutation Service
Premiu m Paymen
Application Components and Services
CRM System
CIS
Policy Data Management
Claim InfoSer
Financial Application
Palette
Navigator
Hints
Business Process
A Business Process represents a sequence of business behaviors that achieves a specific result such as a defined set of products or business services.
A Business Process describes the internal behaviour performed by a Business Role that is required to produce a set of products and services. For a consumer the required behaviour is not of interest so a process is designated "Internal".
A complex Business Process may be an
Register
Properties
Visualiser
Register (Viewpoint Filter: None, Element Filter: All, Relationship Filter: All)
Accept
Damage Claim
Damage Occured
create/ update
Register
Insurance
Claim Registration Service
create/ update
Customer File
Customer data mutation Service
Scripts Manager
examples
lib
Anonymize
Change Concept Type
Console
Create Views and Visual Objects
Export to Excel
Export View Image
Folder Management
Heat
Hello World
New Model with Elements and Relationship
Open Model and list all Concepts

The Zachman Framework for Enterprise Architecture™

The Enterprise Ontology™

Version 3.0

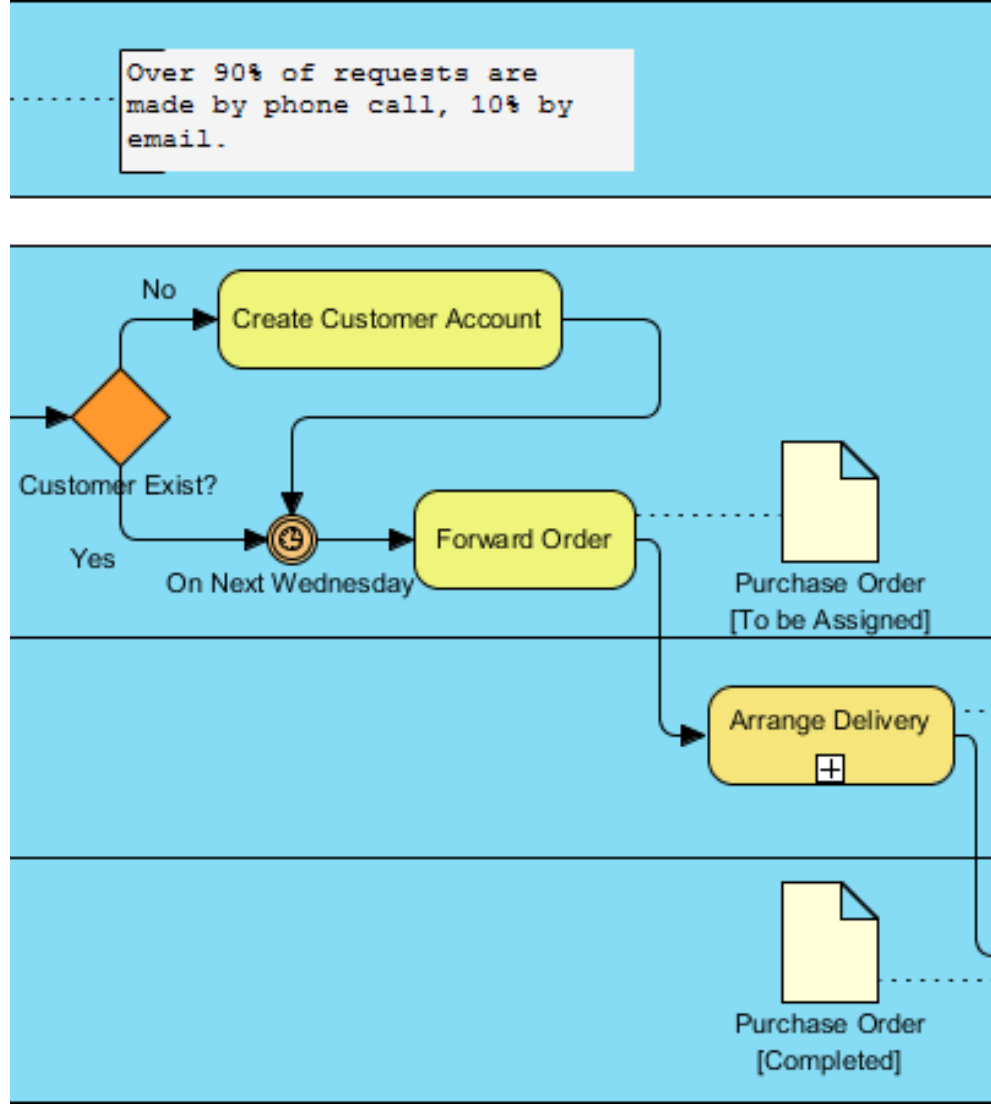


© 1987-2011 John A. Zachman, all rights reserved. Zachman® and Zachman International® are registered trademarks of John A. Zachman
To request Permission Use of Copyright, please contact Zachman.com

BPMN

Business Process Modeling Notation

BPMN (Business Process Model and Notation) diagrams are categorized into four main types: Process Diagram, Collaboration Diagram, Choreography Diagram, and Conversation Diagram. These diagrams are used to model business processes and how they interact.



UML Diagrams

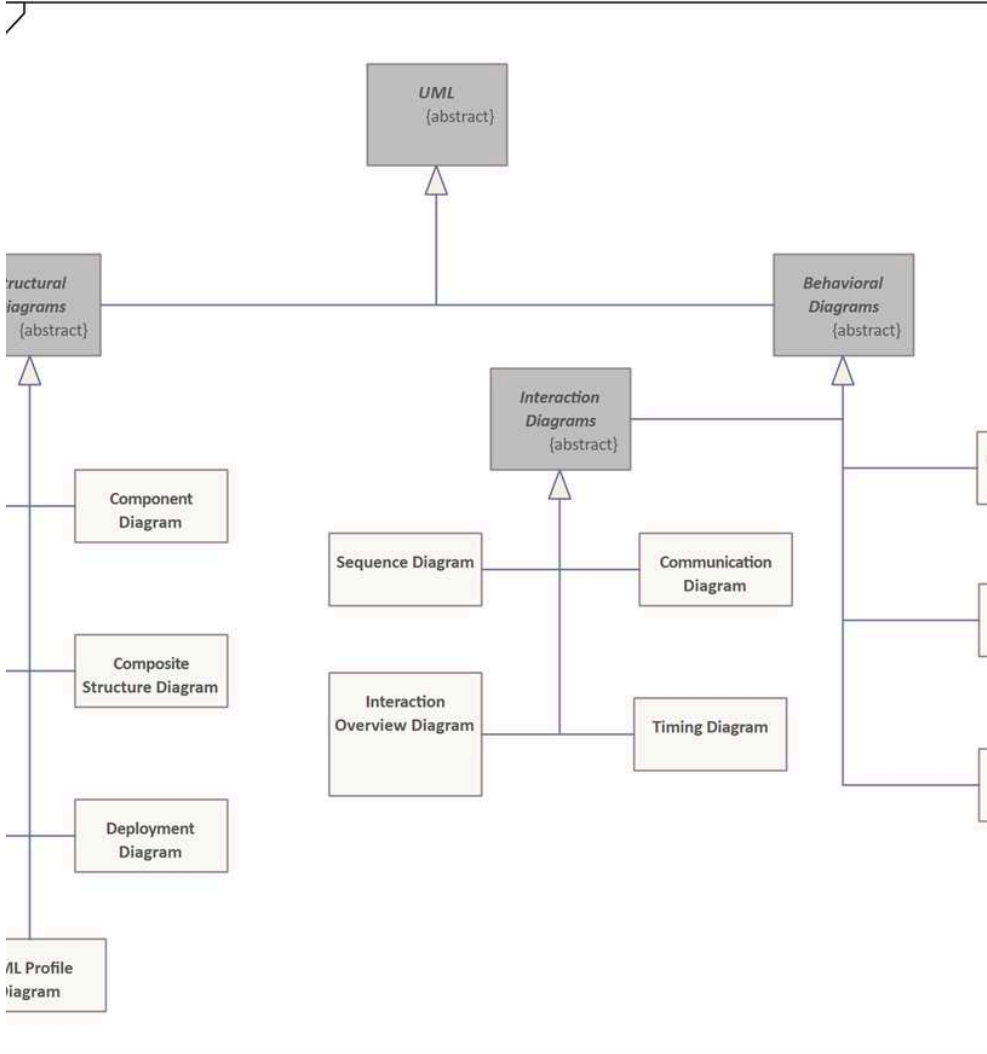
Unified Modeling Language

Structure Diagrams

- Class Diagram
- Component Diagram
- Deployment Diagram
- Object Diagram
- Package Diagram
- Profile Diagram
- Composite Structure Diagram

Behavioral Diagrams

- Use Case Diagram
- Activity Diagram
- State Machine Diagram



Which structures to choose?

- Architectural patterns
- Architectural styles
- Reference architectures

Software (or system) architecture — should be the result of work by a single architect or a small group with a clear technical leader.

The architect (or architecture team) — should continually adapt the architecture to a strictly defined and prioritized list of quality attribute requirements.

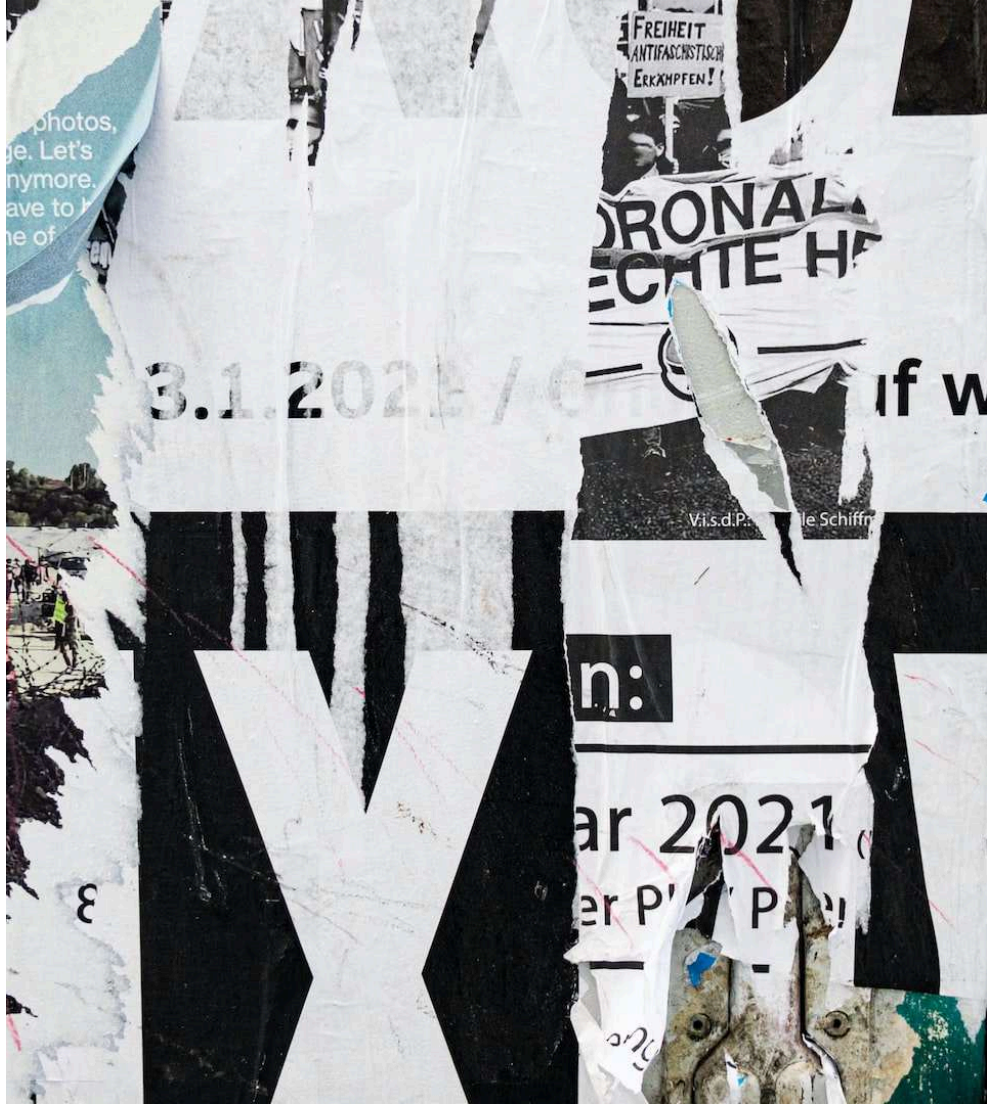
The fewer, the better

Architectural Patterns

Quality attributes should be achieved using proven architectural patterns and techniques.

- Patterns are proven models and views.
- Styles are sets of patterns that work well together.
- Reference architecture is a written set of patterns in a chosen style.

Architecture **should never** depend on a specific version of a commercial product or tool.



Low coupling

Data-producing modules should be separated from data-consuming modules.

Modules and components from different perspectives won’t always correspond one-to-one.

Architecture should apply a small number of simple patterns for component interactions.

Components should be separated based on their responsibilities (SRP — Single Responsibility Principle).

Architecture should include isolated, small areas of resource contention.

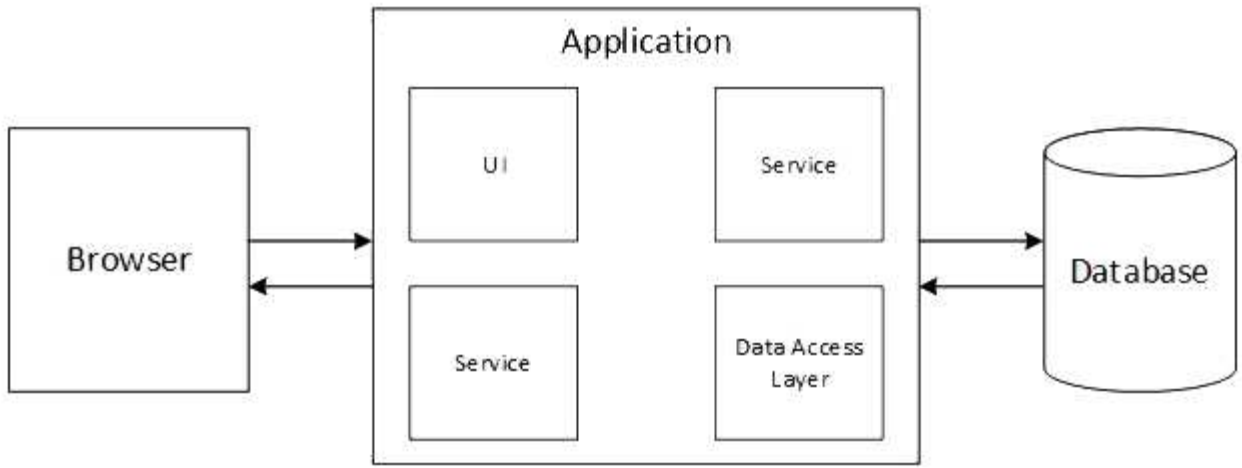


Selected

styles and architectural patterns

Monolithic Architecture

A monolith is an architecture where all operations are performed in a single, undivided system.



All data is collected in a central database, making access easier.

The application has a dedicated machine, which has advantages such as ease of management and monitoring.

However, the drawbacks include limitations in capabilities and the risk of failure of a single machine.

Monolithic Architecture

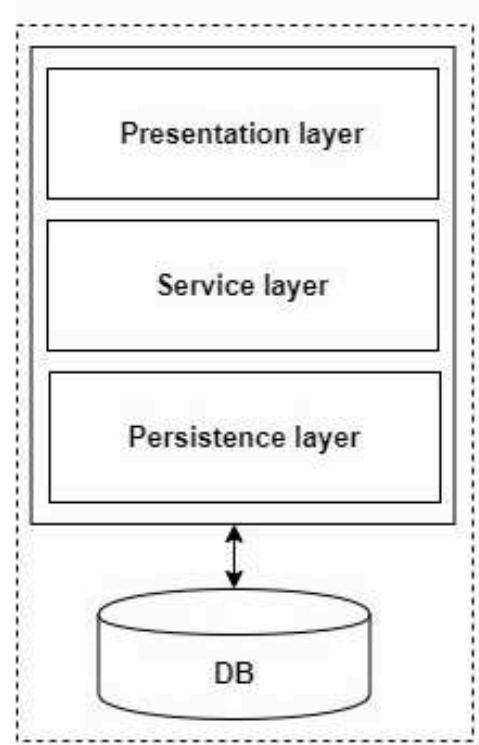
It is a single, unified application that is self-contained and independent of other applications.

It is the simplest, but lacks flexibility.

Alternatives to Monolith:

- Modular monolith
- Multi-layer architectures
- Distributed processing
- Service-oriented architecture
- Microservices
- Serverless

Each of them has a different set of pros and cons.



Monolithic architecture

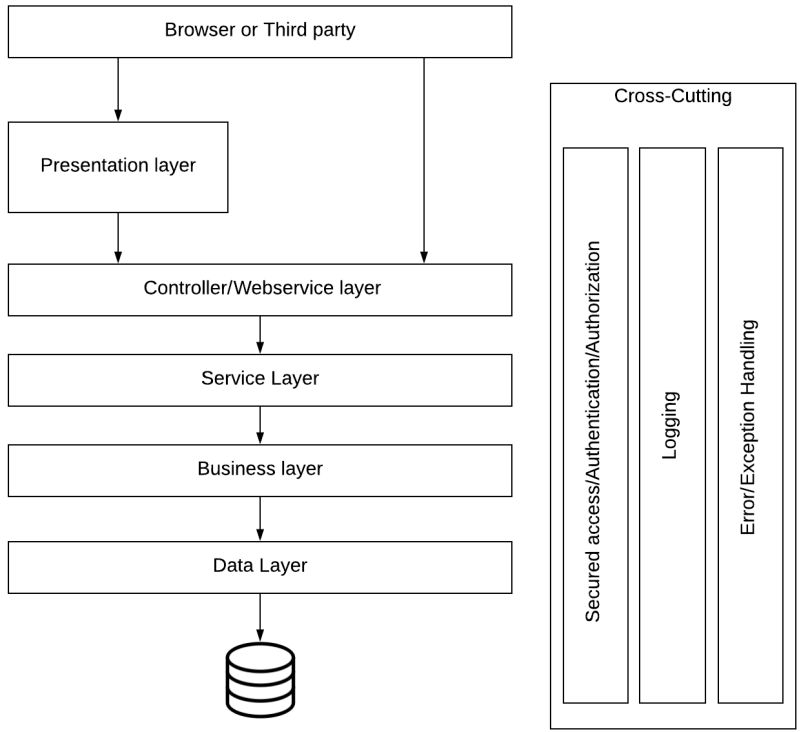
Modular Monolith

Architectures that increase the flexibility of a monolith

- Modules
- Layers
- Distributed monolith

Layers and modules isolate the impact of changes on other parts of the system

(Encapsulation, Open/Closed Principle)



* Design Patterns and Best Practices in Java By Kamalmeet Singh, Adrian Ianculescu, Lucian-Paul Torje

Service-Oriented Architecture

– a system design concept that focuses on defining services to meet user requirements.
(Service-Oriented Architecture, SOA)

The goal of SOA is to link the business side of an organization with its IT resources.

Service

- defines any software component that is independent of others, has a defined interface, and provides a clearly specified set of functions

Interface

- hides implementation details – invisible and irrelevant from the client's perspective.

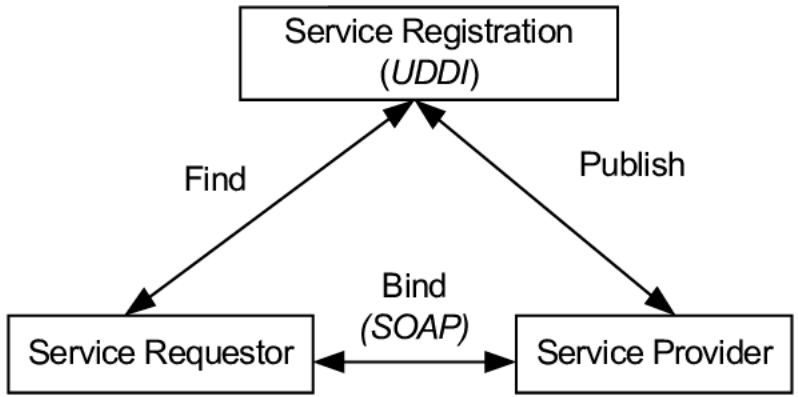
Communication medium

- Shared among all services, enabling data flow between elements of the platform.

BPMN notation can be used to model business processes implemented in SOA.

Service Oriented Architecture

- **consumer** – the client using the services
- **service provider** – a service is the execution of a task via a known interface
- **service registry** – a place where the client obtains information about available services (UDDI)



- Data transmission standards, e.g.:
  - HTTP Protocol, XML Format, JSON
- Ways of describing and invoking a service
  - WSDL (Web Services Description Language)
  - RPC Protocol

Principles of Service Oriented Architecture

Standardized Service Contract – services should be described in a consistent way

Loose Coupling – strive for independence among services

Service Abstraction – services hide/encapsulate their internal logic

Service Reusability – decomposing logic into services enables reuse

Service Autonomy – services must control the logic they encapsulate

Service Statelessness – services should operate only on the state of the current message

Service Discoverability – service repository/registry – for discovery and description of services

Service Interoperability – standards used by a spectrum of different consumers

https://www.opengroup.org/soa/source-book/intro/index.htm

SOA Maturity Levels (OSIMM)

At the heart of OSIMM are seven levels of business and IT service integration maturity.

Level 1: Silos – Different parts of the organization independently develop their own software, without integration of data, processes, standards, or technologies.

Level 2: Integrated – Technologies have been introduced to enable communication between silos and the integration of data and interconnections.

Level 3: Componentized – IT systems within silos are broken down into components, allowing for development into new configurations and systems.

Level 4: Service – Applications are composed of loosely coupled services. Invocation of services is based on open standards and is independent of application technology.

Level 5: Composite Services – At this maturity level, business processes can be created for a set of cooperating services, not just by custom development, but through composition or business process modeling.

Level 6: Virtualized Services – Business and IT services are now provided through a façade – an intermediary layer. The consumer does not invoke the service directly but calls a "virtual service".

SOA Implementation Models

Enterprise Service Bus (ESB)

An intermediary layer in a multi-tier architecture that facilitates the application of SOA concepts in a corporate environment. It allows for dynamic connection and disconnection of services that make up the corporate information system.

Microservices

An architectural pattern that organizes an application as a collection of loosely coupled, fine-grained services communicating via lightweight protocols. It allows teams to develop and deploy their services independently of others.

Service Mesh

A dedicated infrastructure layer that facilitates communication between services or microservices via a proxy server.

Problems with SOA and Monolithic Architecture

A large, monolithic application makes development more complex and slower.

Scalability – individual components cannot be scaled independently.

Reliability – If any module fails, it could impact the availability of the entire application.

Technology adoption barrier – any changes in the framework or language affect the whole application, making changes costly and time-consuming.

Lack of flexibility – The monolith is limited by the technologies already applied in it.

Deployment — a small change in a monolithic application requires redeploying the entire monolith.

Microservice Architecture

Arranges the application as a set of loosely connected small services communicating via lightweight communication protocols.

The goal of microservices is to ensure the independence of individual components, which can be developed **independently** of other parts of the system and have a clear division so that each one performs a specific business or software logic.



Microservices - Advantages

Microservices consist of individual, loosely coupled services that can be developed, deployed, operated, changed, and redeployed without affecting other services or the application's integrity.

Small team responsibility - Microservice development is typically handled by small, independent teams, making it easier to implement Agile and DevOps practices.

Flexible scaling — new instances of a service can be quickly deployed in a cluster to reduce load. Multiple providers and clients distributed across several instances can be used.

Automated infrastructure - Teams developing and maintaining microservices typically use infrastructure automation practices like continuous integration (CI), continuous delivery (CD), and continuous deployment (also CD).

Microservices - More Advantages

Continuous deployment — frequent and faster release cycles. Updates once a week or two to three times a day.

Easier to maintain and test — teams can experiment with new features and roll them back if something fails. This makes code updates easier and shortens the time to market for new features. Additionally, isolating and fixing bugs in individual services is easier.

Independent deployment — since microservices are independent units, they allow for quick and easy independent deployment of individual features.

Technology flexibility – microservice architectures allow teams to freely choose the tools they need.

High reliability – you can deploy changes for a specific service without risking bringing down the entire application.

Happier teams — teams working with microservices are much happier as they are more autonomous and can build and deploy independently without waiting weeks for pull request approval.

Microservices - Disadvantages

More services across more locations created by multiple teams.

Exponential infrastructure costs — each new microservice can incur its own set of tests, deployment guides, hosting infrastructure, monitoring tools, and more.

Added organizational overhead — teams need to add another layer of communication and collaboration to coordinate updates and interfaces.

Challenges with debugging — each microservice has its own set of logs, which complicates debugging. Additionally, a single business process may span multiple computers, further complicating debugging.

Lack of standardization – without a common platform, the number of languages, logging, and monitoring standards can increase.

Lack of clear ownership – as the number of services increases, so does the number of teams managing those services. Over time, it becomes hard to know which services a team can rely on and who to contact for support.

What are Microservices?

Microservices are small, independent, and loosely coupled. They can be deployed independently.

Each service represents a separate codebase that can be managed by a small development team.

Services are independent and responsible for persisting their own data or external state.

Services encapsulate logic and communicate with each other using well-defined APIs.

Services don’t need to use the same technology stack, libraries, or structures.

How to Create Microservices

For many organizations, the starting point is monolithic architecture.

Next steps are:

- Split the codebase into multiple services,
- Implement the right patterns to efficiently handle failures and restore functionality,
- Address data consistency issues,
- Monitor,
- Organize teams (and introduce a DevOps culture).

Reorganizing a monolithic database schema can be a delicate operation. It's important to precisely identify the data sets needed by each service and overlapping areas.

Continuous delivery helps reduce the risk of failed releases and ensures that teams can focus on creating and maintaining the application, not its tedious deployment.

Distributed Architecture

Microservices belong to the category of distributed systems.

A distributed system is a set of programs that use resources from multiple separate computational nodes to achieve a common goal.

Distributed systems often help improve the reliability and performance of a system and make scaling easier.

Nodes in a distributed system provide redundancy—if one node fails, other nodes can take over its function. Each node can be scaled horizontally and vertically, which improves performance.

If the system is under heavy load, more nodes can be added to help absorb the load.

However, distributed architecture adds significant complexity and introduces new types of problems.

Microservice Architecture (Patterns)

Direct communication

API Gateway

- Communicate with services through a common entry point.

Service Registry / Discovery

- Services can register/find each other.

Orchestration

- One of the services acts as the orchestrator - allowing for placing and rebalancing services.

Choreography

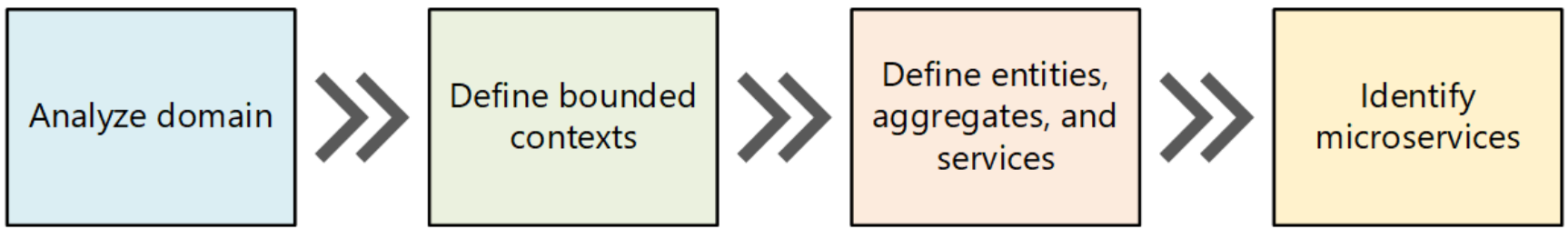
- Services can decide when and how to process operations, instead of relying on the orchestrator.

Tracking and monitoring

Microservice Architecture Development Process

Using Domain Driven Design (DDD)

1. Start by analyzing the business domain to understand the functional requirements of the application.
2. Each bounded context contains a domain model representing a specific subdomain of the larger application.
3. Tactical DDD patterns within the bounded context to define entities, aggregates, and domain services.
4. Use the results from the previous step to identify microservices within the application.



https://learn.microsoft.com/pl-pl/azure/architecture/microservices/model/domain-analysis

Scenario:

Drone Fleet Management

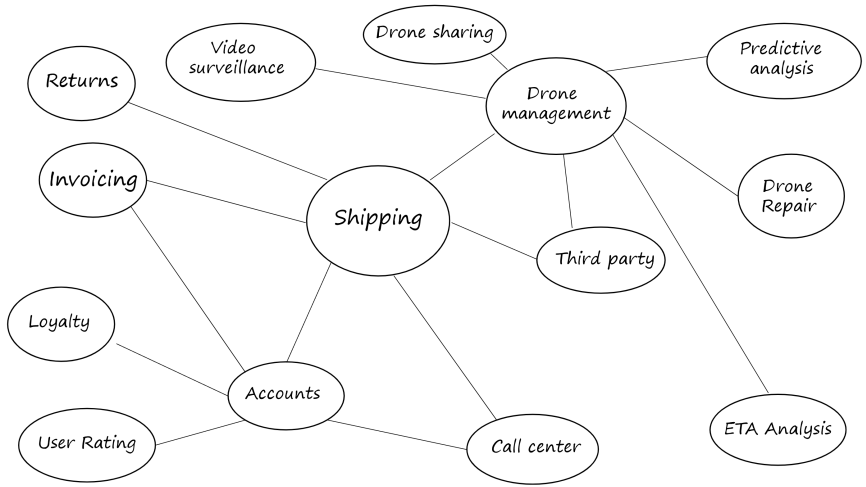
Delivering goods via drones

Companies sign up for the service, and users can order a drone to pick up goods for delivery.

When the client schedules a pickup, the internal database system assigns a drone and notifies the user with the estimated delivery time.

While the delivery is in progress, the client can track the drone's location and the constantly updated estimated delivery time.

https://learn.microsoft.com/pl-pl/azure/architecture/microservices/model/domain-analysis

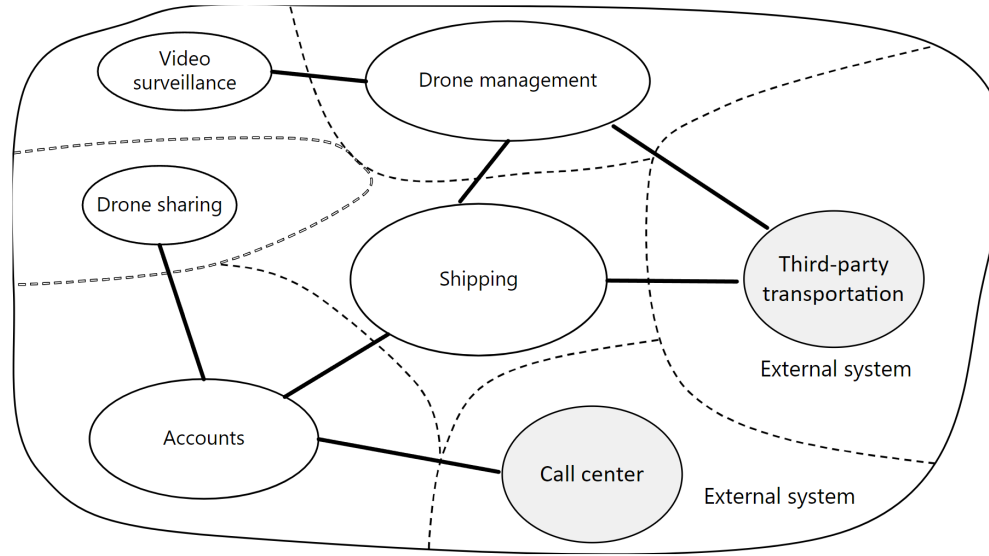


Scenario:

Drone Fleet Management

What is the main domain, and what is the auxiliary?

- Shipping
- Drone management
- ETA analysis provides an estimate for pickup and delivery times.
- Alternative transport - if the package cannot be fully delivered by drone.
- Drone surplus rental
- Video surveillance - optional service
- User accounts, billing



Drone Delivery: Applying Patterns

We start with scenarios that the Shipping bounded context must handle.

- The client can request a drone to pick up goods from a company registered with the drone delivery service.
- The sender generates a tag (barcode or RFID) to be placed on the package.
- The drone picks up and delivers the package from the source to the destination.
- When the client schedules the delivery, the system provides the ETA based on the route, weather, and historical data.
- While the drone is in flight, the user can track the current location and the latest ETA value.
- Until the drone picks up the package, the client can cancel the delivery.
- The client is notified upon completion of the delivery.
- The sender can request delivery confirmation from the client, either through a signature or fingerprint.
- Users can search the history of completed deliveries.

DDD Patterns: Value Objects, Entity Aggregates (Instances), Domain Services, Events, etc.

https://learn.microsoft.com/pl-pl/azure/architecture/microservices/model/tactical-ddd

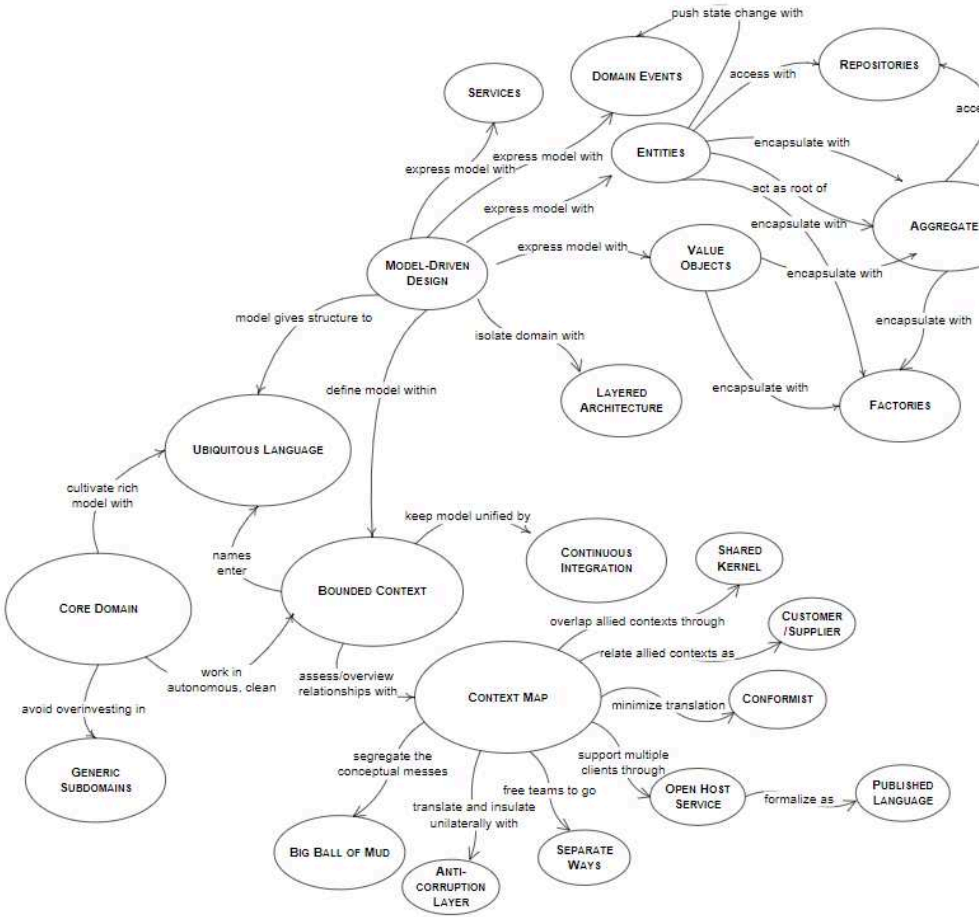
Domain Driven Design

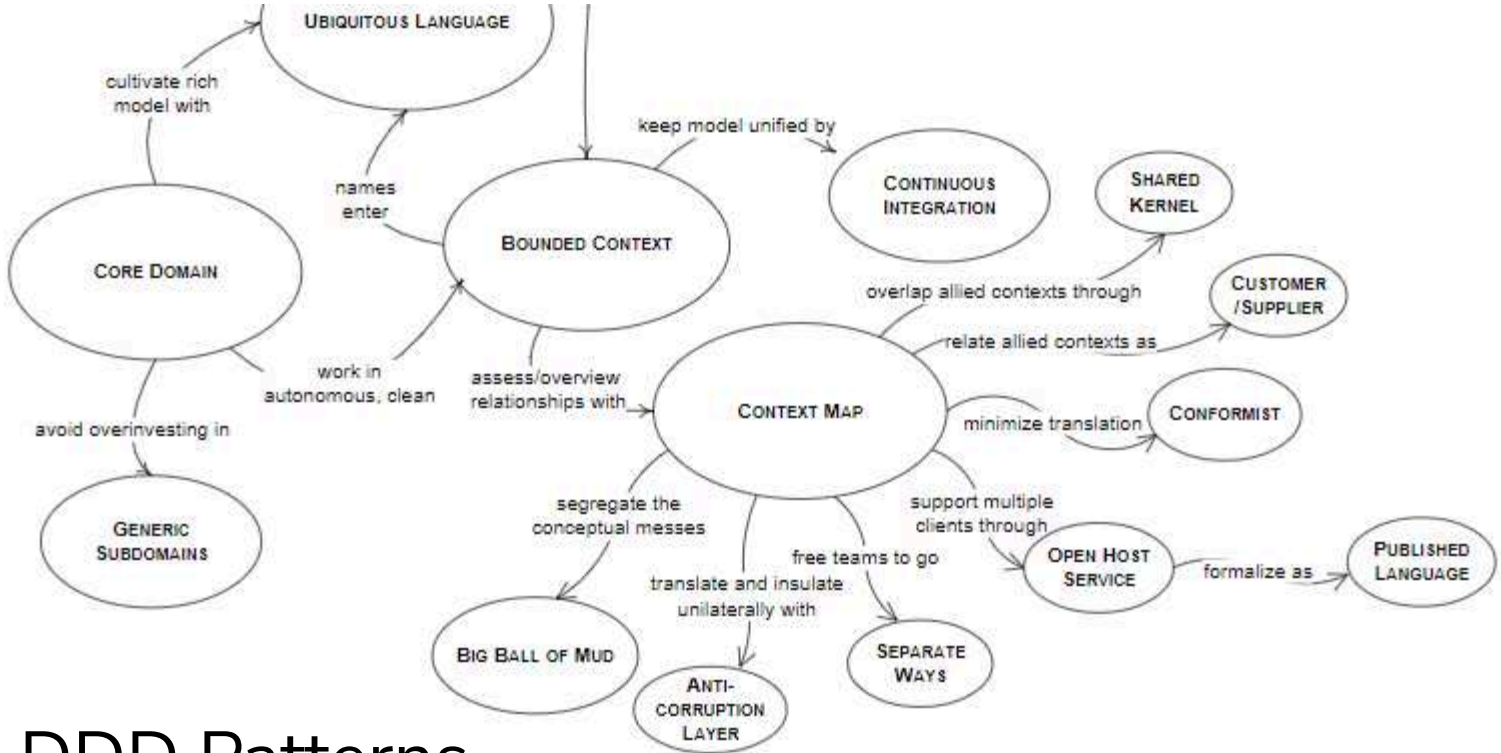
is a software design approach that focuses on modeling the domain
based on the knowledge of experts in a given field.

Domain – a specific context for the problem we want to solve

In domain-driven design, the structure and language of the software code (class names, method names, variable names) should reflect the business domain.

For example, a lender's software may include classes like **"loan application"**, **"customer"**, and methods such as **"accept offer"** and **"withdraw application"**.





Strategic DDD Patterns

used to define bounded contexts – logical and physical boundaries around cohesive domains

- Division into core, generic, and supporting (sub-) domains
- Ubiquitous domain language – a shared, unambiguous vocabulary for all stakeholders

DDD Strategic Design

Strategic design is about organizing large systems by dividing the domain into cohesive bounded contexts.

It focuses on:

- Team boundaries
- Language consistency
- Integration between subsystems

Bounded Context

A bounded context is an explicit boundary within which a domain model is defined and applicable.

Key Characteristics:

- Owns its model, language, and logic
- Has clear integration points with other contexts
- Maps closely to a team or subdomain

Examples:

- In an e-commerce system:
  - Shipping Context
  - Billing Context
  - Catalog Context

Each has its own model of "Order" suited to its needs.

They share concept, but have different implementations of it.

Separate contexts

but the same, shared concepts - https://martinfowler.com/bliki/BoundedContext.html

Context Map

A context map defines relationships and integration patterns between bounded contexts.

Common Integration Patterns:

- Shared Kernel
- Customer/Supplier
- Conformist
- Anti-Corruption Layer
- Open Host Service
- Published Language

Example:

- Billing context is a Customer of the Catalog context (Supplier)
- The UI context uses an Anti-Corruption Layer to translate calls to the Billing model

How Politics Affect Microservice Design

- **Politics: Separate Ways**
  - If integration is too costly (technically or politically), integration may be skipped entirely.
  - Manual processes can replace automated integration (e.g. manually creating users in separate systems).
  - This decision should be documented in the **Context Map** to ensure visibility across teams.
- **Politics: Partnership**
  - Teams responsible for different contexts but with shared goals can form partnerships.
  - Partnerships support tighter integration through:
    - Aligned release schedules
    - Shared kernel or components
    - Mutual planning and cooperation

How Politics Affect Microservice Design

- **Politics: Upstream / Downstream** (Supplier /Customer)
  - Microservice teams can be categorized as **upstream** (supplier) or **downstream** (consumer).
  - The downstream context depends on the upstream's data, APIs, or behavior.
  - This directional relationship informs communication and planning structures.
  - A contract or agreement is established between teams.
  - Downstream team members may attend upstream meetings to align priorities and timelines.
- **Conformist**
  - The upstream service does not accommodate client-specific needs.
  - The downstream team must adapt to the upstream's interface (common with third-party services).
  - Example: A payment provider exposes a fixed API that clients must conform to.

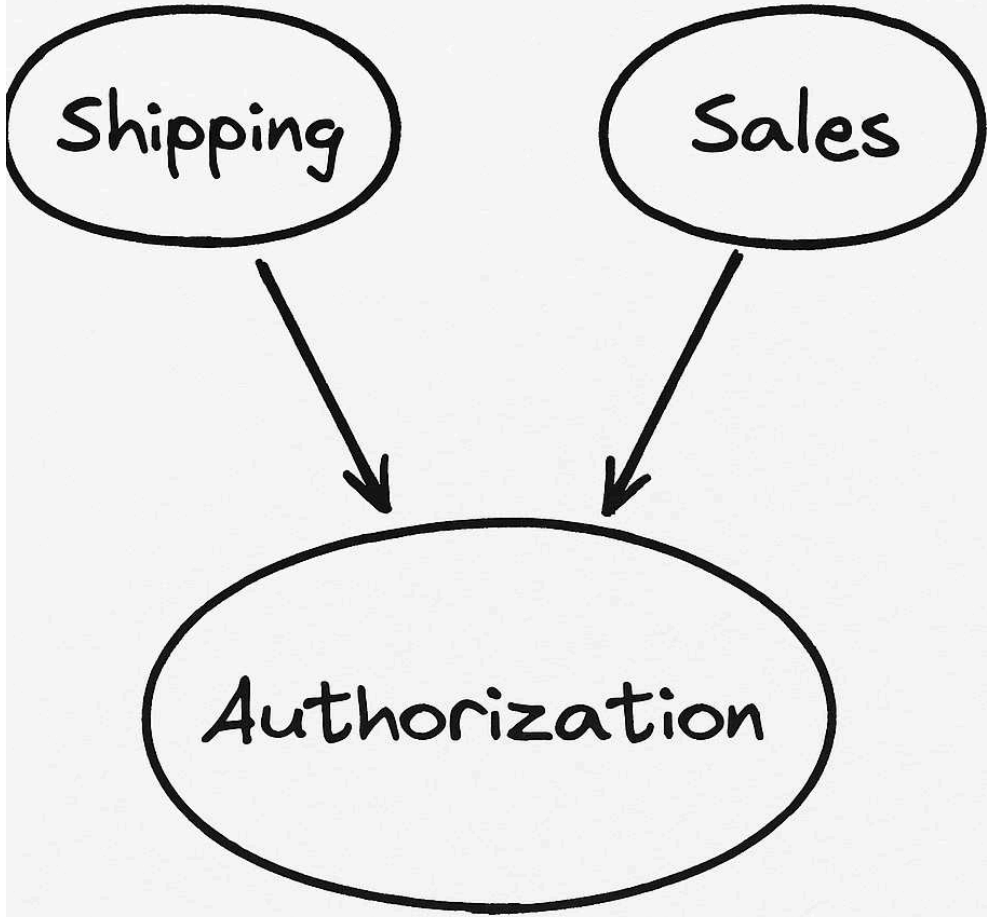
Shared Kernel

Two contexts share a subset of the domain model and must coordinate changes.

Use with caution due to tight coupling.

Example:

- Authentication module shared between User Profile and Admin Panel



Customer / Supplier

One context (Customer) depends on another (Supplier) to provide functionality or data.

Example:

- Invoice Generation context (Customer)
- Accounting Rules context (Supplier)

The Customer adapts to the Supplier’s model and schedule.

Conformist

The Customer context conforms to the Supplier’s model as-is.

Trade-off:

- Fast integration
- Little control or customization

Example:

- A reporting system consuming raw data from Sales without transforming it

Anti-Corruption Layer (ACL)

A layer that translates between models to protect the integrity of a bounded context.

Example:

- CRM integrates with external ERP system through ACL:
  - Translates ERP "Client" to CRM "Customer"
  - Prevents leaking foreign logic into CRM

Open Host Service & Published Language

- Open Host Service: Expose a well-defined API to other contexts.
- Published Language: A shared, agreed-upon protocol or schema.

Example:

- Shipping context exposes a REST API
- Uses OpenAPI spec or Protobuf as the Published Language

Summary

- Strategic patterns guide how parts of the system interact.
- Bounded contexts provide modularity and clarity.
- Use context maps to explicitly document relationships.
- Choose integration styles based on team dynamics and control needs.

Bounded contexts are the map.

Integration patterns are the tools.

Tactical DDD Patterns

Patterns used by developers, but with terminology understandable to stakeholders

- Entities – A class with an identity independent of its values; tracks and persists changes to its data
- Value Objects – An immutable class whose identity is defined by a combination of its values
- Repository – encapsulates persistence for aggregates
- Aggregate – a transactional graph of objects
- Domain events, side effects
- Factories and domain services

Entities

```
class User {
  constructor(
    public readonly id: string,
    public name: string,
    public email: string
  ) {}

  changeEmail(newEmail: string) {
    this.email = newEmail;
  }
}
```

Here, User is an entity. Even if name or email changes, its identity (id) remains constant.

- An entity has a distinct identity that runs through time and different states.
- Changes to its properties do not affect its identity.

Exercise:

List candidates for Entity in your domain.

Value Objects

Value objects are immutable and are compared by their values, not identity.

```
class Money {
  constructor(
    public readonly amount: number,
    public readonly currency: string
  ) {}

  add(other: Money): Money {
    if (this.currency !== other.currency)
        throw new Error("Currency mismatch");
    return new Money(
        this.amount + other.amount,
        this.currency
    );
  }

  equals(other:Money): boolean { /* ... */ }
}
```

No conceptual identity, just a combination of values.

Two Money instances with the same amount and currency are considered equal.

Exercise:

List candidates for Value objects in your domain.

Repository

A repository provides access to entities and aggregates.

Abstracts away the details of persistence.

```
interface UserRepository {
  findById(id: string): Promise<User | null>;
  save(user: User): Promise<void>;
}
```

This hides persistence logic and allows usage like `repo.findById("abc")`.

Exercise:

List candidates for Repository in your domain.

Aggregates

An aggregate is a cluster of domain objects treated as a single unit.

```
class Order {
  constructor(
    public readonly id: string,
    private items: OrderItem[]
  ) {}

  addItem(item: OrderItem) {
    this.items.push(item);
  }

  getTotal(): number {
    return this.items.reduce((sum, item) => {
        return sum + item.totalPrice()
    }, 0);
  }
}
```

Exercise:

List candidates for your aggregate and its entities.

Has a root entity called the aggregate root.

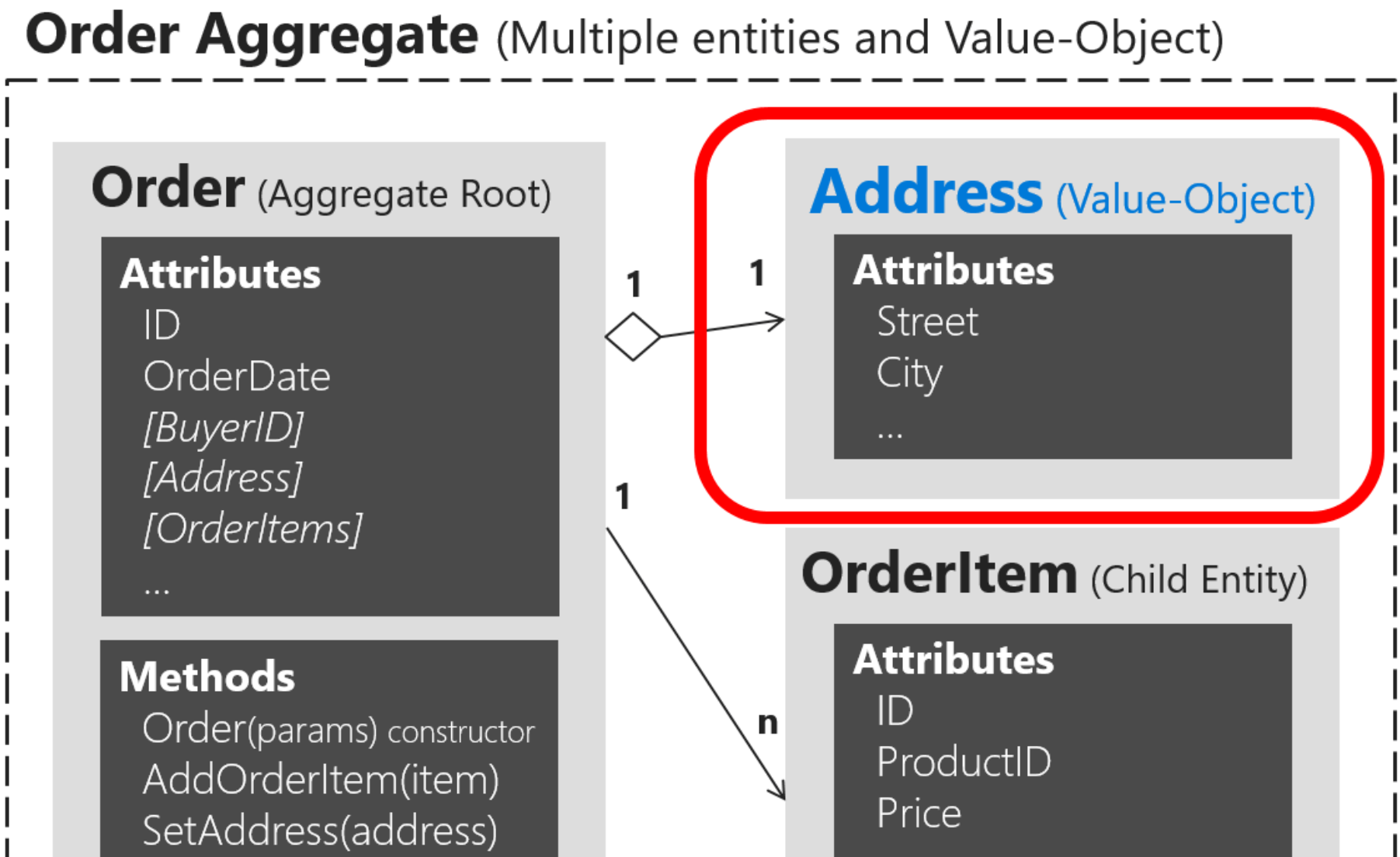
Order is the aggregate root.

```
class OrderItem {
  constructor(
    public productId: string,
    public price: number,
    public quantity: number) {}

  totalPrice(): number {
    return this.price * this.quantity;
  }
}
```

OrderItems can’t be modified independently of aggregate.

Aggregate is the transaction boundary. Its loaded and saved as a whole.


Order Aggregate (Multiple entities and Value-Object)
Order (Aggregate Root)
Attributes
ID
OrderDate
[BuyerID]
[Address]
[OrderItems]
...
Methods
Order(params) constructor
AddOrderItem(item)
SetAddress(address)
1
1
Address (Value-Object)
Attributes
Street
City
...
1
n
OrderItem (Child Entity)
Attributes
ID
ProductID
Price

Factories and Domain Services

Factories create complex domain objects or aggregates.

Factories handle creation - Example:

```
class OrderFactory {
  static createNewOrder(
    customerId: string,
    items: OrderItem[]
  ): Order {
    return new Order(uuid(), items);
  }
}
```

Exercise:

List candidates for services and factories.

Domain services encapsulate domain logic that doesn't naturally belong to any entity or value object.

Services handle behavior that spans multiple domain objects. Example:

```
class PaymentService {
  constructor(
    private paymentGateway: PaymentGateway
  ) {}

  processPayment(
    order: Order,
    paymentDetails: PaymentDetails
  ) {
    const total = order.getTotal();

    return this
            .paymentGateway
            .charge(paymentDetails, total);
  }
}
```

Domain Events & Side Effects

Domain events describe something important that happened in the domain.

```
class UserRegistered {
  constructor(public readonly userId: string, public readonly email: string) {}
}

// Usage
const event = new UserRegistered(user.id, user.email);

eventBus.publish(event);
```

Used to trigger side effects or communicate across bounded contexts.

This captures a meaningful domain change that can be handled elsewhere (e.g., send welcome email).

Exercise:

List candidates for domain events and their recipients.

Domain Events

Separating communication from the domain model.

“Use domain events to capture the occurrence of something that happened in the domain.”

— Vaughn Vernon

Designing Domain Events

- Each event has its own class
- Timestamp – when the event occurred
- Capture details specific to the event
- Event fields are initialized in the constructor
- Objects are immutable and read-only
- No behavior or side effects

Example Events

- User authenticated
- Appointment confirmed
- Payment received

Event Storming

Event Storming is a workshop-based method for quickly exploring what is happening in a software domain.

Compared to other methods, it is remarkably lightweight and intentionally does not require computer support.

The outcome is expressed in sticky notes on a wide wall.

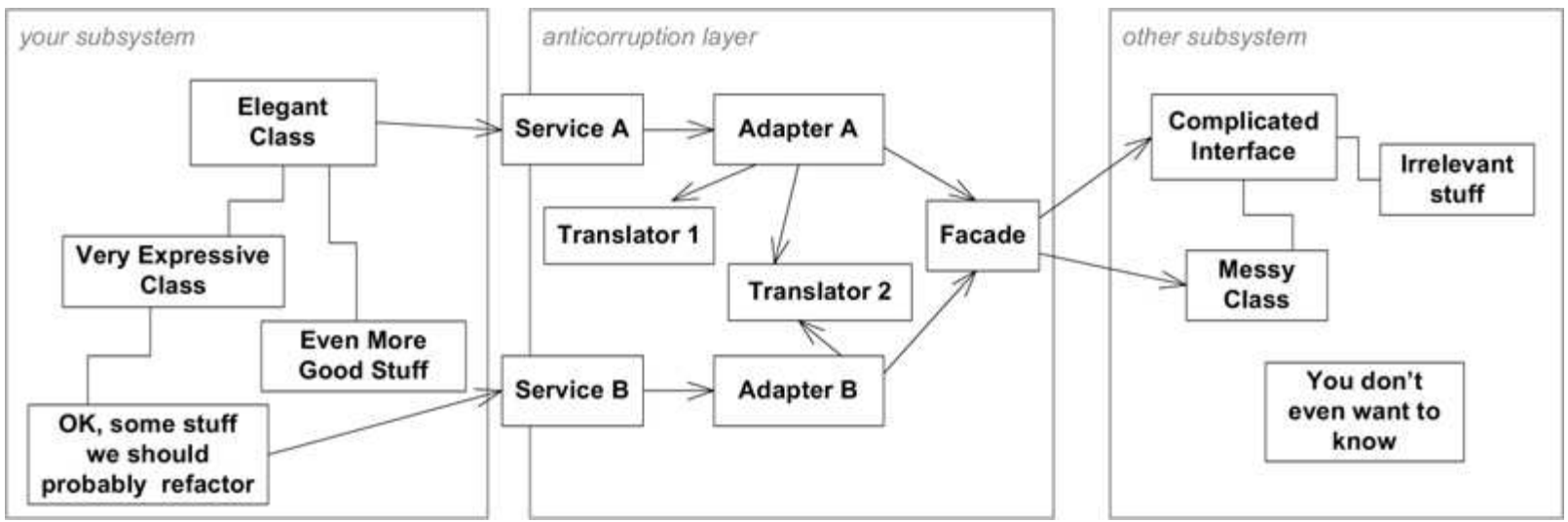


DDD Integration Patterns

Hollywood Principle – “Don’t call us, we’ll call you.”

Inversion of Control (IoC) – A pattern for loosely coupling a dependent object with the object it needs at runtime.

Anti-Corruption Layer (ACL) – Isolates a bounded context and handles interaction with foreign systems or contexts.



Microservice Integration

Synchronous (Request-Response)

- Protocols: HTTP/REST, gRPC
- Easy to implement
- Tighter coupling & higher latency

Example - REST API:

- GET /inventory/
- POST /payment/charge

We need to know when to request the data

Microservice Integration

Asynchronous (Event-Driven)

- Message Brokers: Kafka, RabbitMQ, NATS
- Loose coupling, resilient
- Eventual consistency

Example Events:

- OrderPlaced
- PaymentConfirmed
- ItemShipped

We have to wait for data to come to us

Choosing the Right Approach

Feature	Synchronous	Asynchronous
Coupling	Tight	Loose
Latency	Low (ideal path)	High (eventual)
Error Handling	Immediate	Retry/Dead-letter
Observability	Simpler	Requires tracing tools

Combine both styles for robust, scalable systems.

Lightweight APIs

- JSON – JavaScript Object Notation – a lightweight text-based data exchange format.
- REST – Representational State Transfer – a software architecture style derived from experience in designing the HTTP protocol specification for distributed systems.

REST incorporates concepts such as:

- Uniform interface
- Stateless communication
- Resources and their representations
- Hypermedia (Links)
- HATEOAS
  - Hypermedia As The Engine Of App State


```
- {
    userId: 1,
    id: 2,
    title: "quis ut nam facilis et officia qui",
    completed: false
  },
- {
    userId: 1,
    id: 3,
    title: "fugiat veniam minus",
    completed: false
  },
- {
    userId: 1,
    id: 4,
    title: "et porro tempora",
    completed: true
  },
- {
    userId: 1,
    id: 5,
    title: "laboriosam mollitia et enim quasi adipisci quia provident illum",
    completed: false
  },
```

```yaml
  version: 1.0.0
servers:
  - url: https://api.swagger.io
paths:
  /resources:
    get:
      tags:
        - API Resources
      summary: Retrieve all resources based on filter criteria
      description: Fetches a list of all available resources.
      responses:
        '200':
          description: A
          content:
            application/j
              schema:
                type: arr
                items:
                  $ref: '
    post:
      tags:
        - API Resources
      summary: Create a n
      description: Create
      requestBody:
        required: true
        content:
          application/jso
            schema:
              $ref: '#/co
```

Servers

https://api.swagger.io

API Resources

GET	/resources	Retrieve all resources based on filter criteria
POST	/resources	Create a new resource
GET	/resources/{resourceId}	Retrieve a resource
PUT	/resources/{resourceId}	Update a resource
PATCH	/resources/{resourceId}	Partially update a resource
DELETE	/resources/{resourceId}	Delete a resource
OPTIONS	/resources/{resourceId}	Retrieve options
HEAD	/resources/{resourceId}	Check resource presence

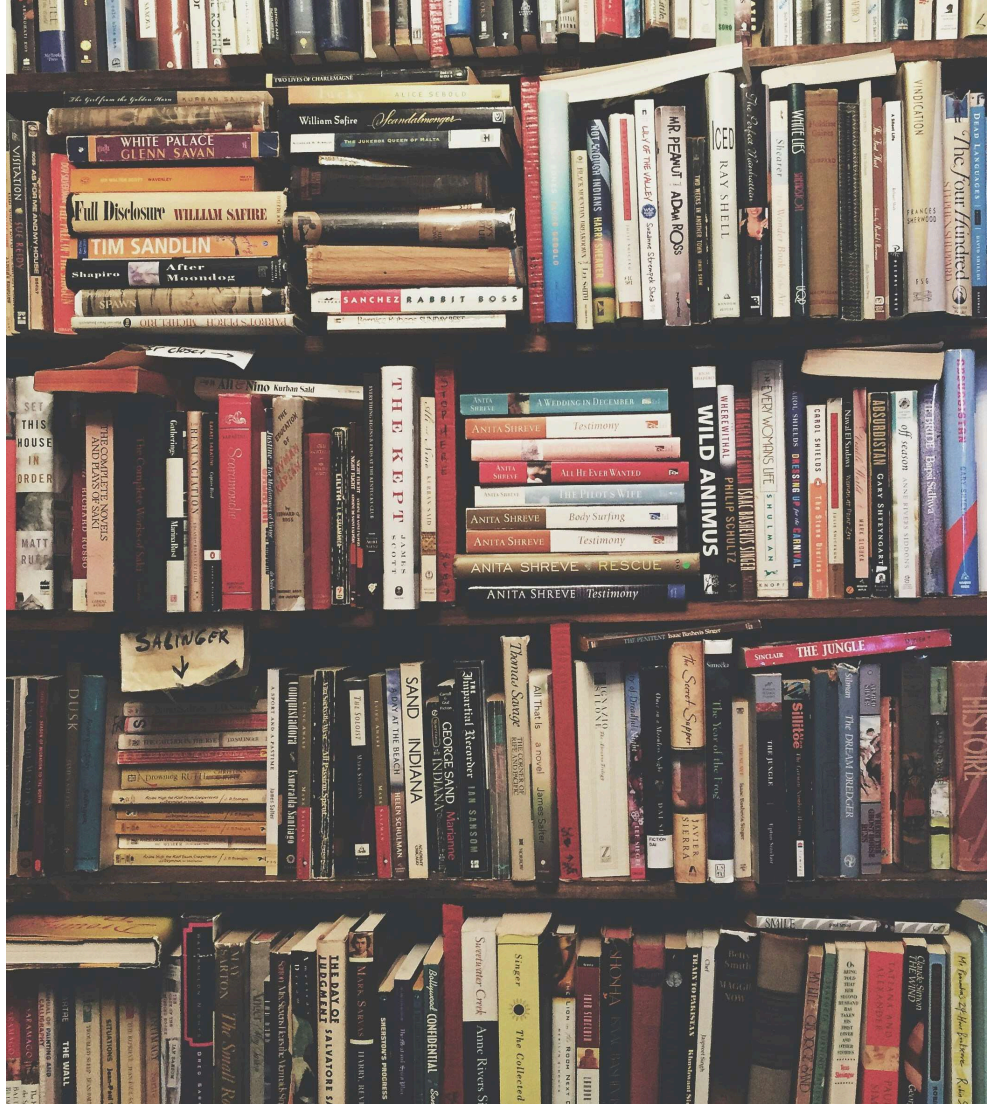
Bookstore API

Design a RESTfull API

1. View a list of all books (+ filters)
2. View details of a specific book
3. Add a new book
4. Update a book's details
5. Delete a book
6. Add a review to a book
7. List all reviews for a book

Requirements:

- Use standard HTTP verbs :
  - GET, POST, PUT/PATCH, DELETE
- Keep resources and URLs intuitive and plural
- Follow REST conventions



Not every app is

Cloud Native

https://12factor.net/

12-factor methodology

Twelve aspects of an application in the form of a software-as-service (SaaS) or web applications

- **Use a declarative format** to automate application configuration in order to reduce the time and costs of onboarding new developers to the project;
- **Have a clear contract with the operating system**, enabling as much portability as possible between the environments in which they operate;
- **Are tailored for deployment on modern cloud computing platforms**, preventing the need for servers and system administration;
- **Minimize discrepancies between development and production environments**, enabling continuous deployment of the application to maximize the speed of changes;
- **Can scale without major changes to tools**, architecture, or team workflows.

The twelve-factor methodology can be applied to applications written in any programming language and utilizing any combination of supporting services (databases, queues, caches, ...).

12 factor

12 factors for better service-based applications (SaaS)

- I. Codebase -
  One codebase tracked in version control, many deploys
- II. Dependencies -
  Explicitly declare and isolate dependencies
- III. Configuration -
  Store configuration in the environment
- IV. Backing services -
  Treat backing services as attached resources
- V. Build, release, run -
  Separate the build, release, and run stages
- VI. Processes -
  Run the app as one or more stateless processes
- VII. Port binding -
  Export services via port binding
- VIII. Concurrency -
  Scale out via the process model
- IX. Disposability -
  Maximize robustness with fast startup and graceful shutdown
- X. Dev/prod parity -
  Keep development, staging, and production as similar as possible
- XI. Logs -
  Treat logs as event streams
- XII. Admin processes -
  Run admin/management tasks as one-off processes

1. Codebase

A twelve-factor app is always tracked in a version control system and has a single codebase for each app.

Example:

Store all application code in a single Git repository per service.

Tool:

Git, GitHub, GitLab, Bitbucket

Benefits:

Clear traceability, simpler deployments, collaboration-friendly.

2. Dependencies

Explicitly declare and isolate dependencies using a dependency manager.

Example:

Use package.json to declare all Node.js dependencies.

Tool:

npm, pip, Maven, Bundler

Benefits:

Reproducible builds, reduced deployment errors.

3. Config

Store configuration in the environment instead of the codebase.

Example:

Set database URL via an environment variable DATABASE_URL.

Tool:

dotenv, Kubernetes Secrets, AWS SSM Parameter Store

Benefits:

Safer configuration changes, easier separation of environments.

4. Backing Services

Treat backing services (databases, queues, caches) as attached resources.

Example:

Connect to a PostgreSQL database via an environment variable, not hardcoded config.

Tool:

Heroku Add-ons, Docker Compose, service bindings

Benefits:

Easy swapping of services, better portability.

5. Build, Release, Run

Strictly separate build, release, and run stages of deployment.

Example:

Build a Docker image once, then deploy with environment-specific configs.

Tool:

Docker, CI/CD pipelines (GitHub Actions, Jenkins, GitLab CI)

Benefits:

Repeatable deployments, reduced risk of bugs in production.

6. Processes

Run the app as one or more stateless processes; keep data in a backing service.

Example:

Store session data in Redis instead of in-memory.

Tool:

Express.js with Redis store, Spring Boot with external cache

Benefits:

Horizontal scalability, resilience to process restarts.

7. Port Binding

Export services via port binding instead of relying on external web servers.

Example:

A Node.js app listens on process.env.PORT rather than Apache/Nginx.

Tool:

Express, Flask, FastAPI, Spring Boot

Benefits:

Self-contained apps, deployable anywhere.

8. Concurrency

Scale out via the process model using multiple instances of the app.

Example:

Run multiple worker processes to handle background jobs.

Tool:

Kubernetes (replicaSets), Docker Compose, PM2

Benefits:

Predictable scaling, fault isolation, better resource use.

9. Disposability

Make processes fast to start and graceful to shut down.

Example:

Handle SIGTERM to clean up connections before shutdown.

Tool:

Node.js process signals, Docker ENTRYPOINT scripts

Benefits:

Rapid scaling, smoother deployments, faster recovery.

10. Dev/Prod Parity

Keep development, staging, and production environments as similar as possible.

Example:

Use the same Docker image and config format across all environments.

Tool:

Docker, Kubernetes, Terraform

Benefits:

Fewer bugs due to environment drift, consistent behavior.

11. Logs

Treat logs as event streams; write to stdout/stderr, not to files.

Example:

Console.log output gets collected by the platform (e.g., Heroku, Docker, Kubernetes).

Tool:

Fluentd, Logstash, Grafana Loki

Benefits:

Centralized logging, real-time monitoring, easier debugging.

12. Admin Processes

Run administrative or one-off management tasks in the same environment as the app.

Example:

Run a DB migration with the same container image using a CLI command.

Tool:

Rails console, Django manage.py, one-off Kubernetes Job

Benefits:

Safe admin operations, consistent tooling, no need for separate scripts.

DevOps

The term was created from the combination of the words "development" and "operations."

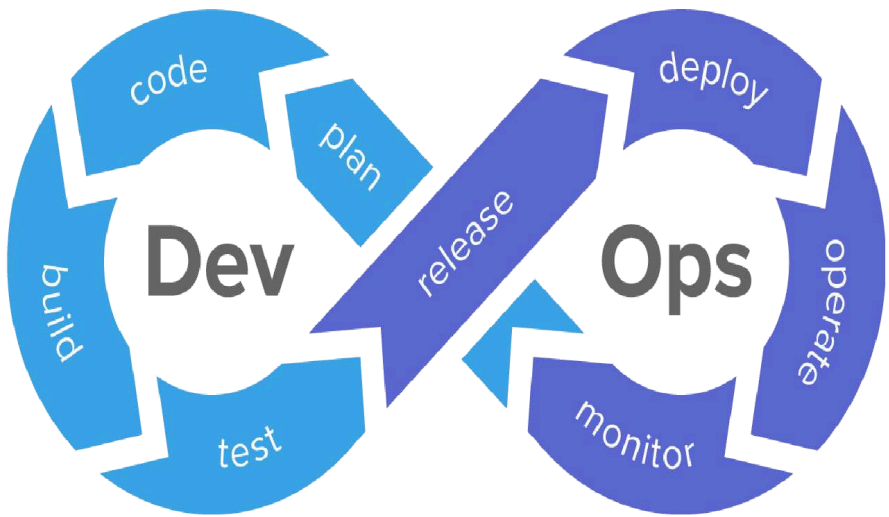
DevOps enables previously separate roles—development teams, IT operations, quality engineering, and security—to coordinate and collaborate to build better, more reliable products.

Benefits of the DevOps methodology

- Faster time to market
- Adaptation to market and competition
- Maintaining system stability and reliability
- Improved mean time to recovery

DevOps Practices

- Configuration Management
- Version Control (GIT)
- Infrastructure as Code
- Continuous Integration and Continuous Delivery (CI/CD)
- Continuous Monitoring
- Agile Software Development

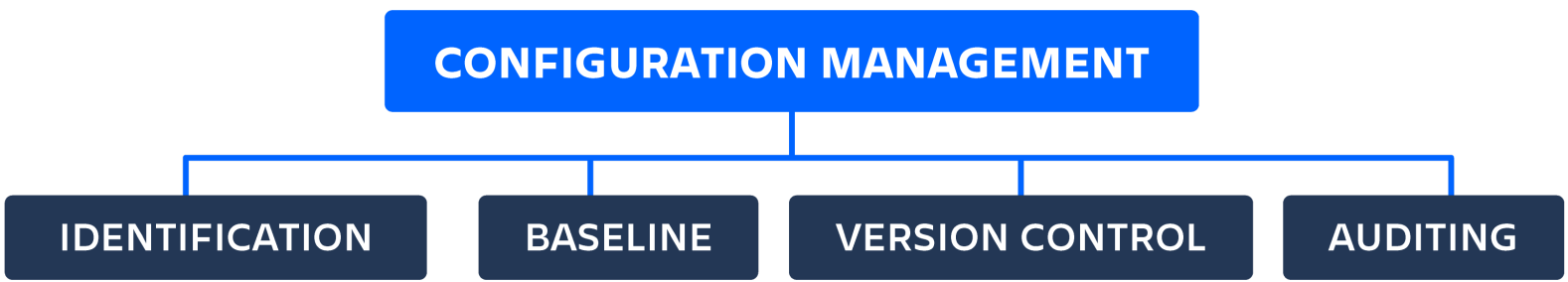


What is Configuration Management?

It's a systems engineering process used to ensure the consistency of product attributes throughout its lifecycle.

- Specifications for computational hardware resource allocations such as CPU, RAM, etc.
- Endpoints specifying external connections to other services, databases, or domains.
- Secret information such as passwords and encryption keys.

Git is an excellent platform for managing configuration data (GitOps).



Configuration Management

The process of managing system and application settings and configurations.

Tools:

Chef, Puppet, SaltStack, Ansible

Benefits:

- Standardized configurations
- Automated system setup
- Faster deployment and changes

Practices:

- Declarative configuration definitions
- Idempotent configuration logic
- Centralized management and auditing

Configuration Management Tools

Git - the most popular version control system used for tracking code changes. Adding configuration management data to a Git repository along with code enables comprehensive version control across the entire project.

Docker - introduces the concept of containerization, which is an advanced form of configuration management. Docker uses configuration files called Dockerfiles, containing a list of commands that are evaluated to recreate the expected system snapshot.

Terraform - an open-source configuration management platform by HashiCorp. This tool uses IaC (Infrastructure as Code) to provision and manage clusters, cloud infrastructure, or services. Terraform supports AWS, Microsoft Azure, and other cloud platforms.

Ansible, Salt Stack, Chef, Puppet - Automate many traditional system administration processes. Each uses a series of configuration data files—usually in YAML or XML format—that are parsed by an executable file.

Deployment

Application deployment methods:

- Manual deployment
- Automation (CI/CD)
- Containerization (Docker)
- Kubernetes

Infrastructure as Code (IaC)

Managing and provisioning infrastructure through code.

Tools:

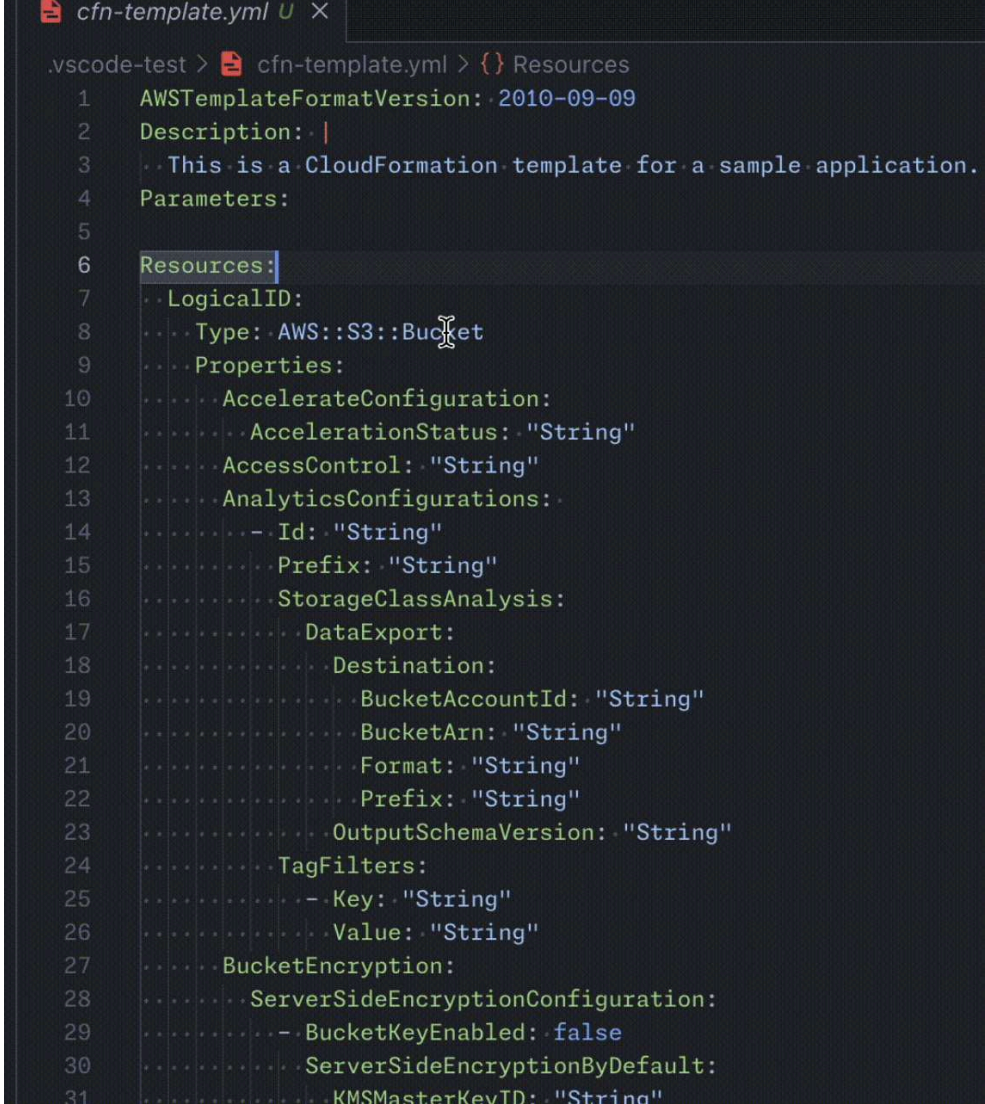
Terraform, AWS CloudFormation, Ansible

Benefits:

- Automated deployments
- Consistency and repeatability
- Version-controlled infrastructure

Practices:

- Use of source code repositories
- Infrastructure testing
- Implementing CI/CD for infrastructure



CI / CD

Continuous Integration and Continuous Delivery

Configuration management – managing the state of resources in the system, including servers, virtual machines, and databases.

- Teams can deploy changes in a controlled and systematic way, reducing the risk associated with modifying system configuration.
- Teams can track system state and avoid configuration drift—situations where a resource's configuration diverges over time from the desired state defined for that resource.

Used alongside the "infrastructure as code" approach, both configuration and system definition enable easy templating and automation, helping teams manage complex environments at scale.

Continuous Monitoring

Ongoing monitoring of systems and applications to ensure performance and availability.

Tools:

Prometheus, Nagios, Grafana, ELK Stack

Benefits:

- Early detection of issues
- Proactive performance management
- Faster failure response

Practices:

- Defining metrics and KPIs
- Setting up alerts
- Regular data reviews and analysis

GIT

GIT is a distributed version control system that allows tracking changes in source code and enables collaboration of many developers on a single project. It was created by Linus Torvalds in 2005.

Advantages of GIT

Distributed: - Each user has a full copy of the repository, ensuring data security and enabling offline work.

Performance: - GIT is fast in operations like commit, merge, and history browsing.

Flexibility: - Supports various workflows, including branches and different work models.

Free: - It is open-source, meaning it can be used for free and adapted to specific needs.

History tracking: - Detailed tracking of the change history for every file in a project.

Branch and merge support: - Enables easy creation and merging of branches, facilitating feature development and team collaboration.

Use Cases of GIT

Software Development:

- Tracking source code changes, version management, and project collaboration.

Team Collaboration:

- Enables simultaneous work of multiple developers on a single project without conflicts.

Open-source Projects:

- GIT is widely used in open-source projects, e.g., on the GitHub platform.

Documentation Management:

- Ability to track changes in text files and project documentation.

Micrcoservice

Design Patterns

Microservice Granularity

Identifying microservice boundaries

- Based on business requirements, not technical concerns
- Service boundaries should not cause data consistency or integrity issues
- Should have high functional cohesion
- No unnecessary inter-service communication
- One responsibility, one role
- Should be loosely coupled
- Can evolve independently

Microservice Pattern Catalog

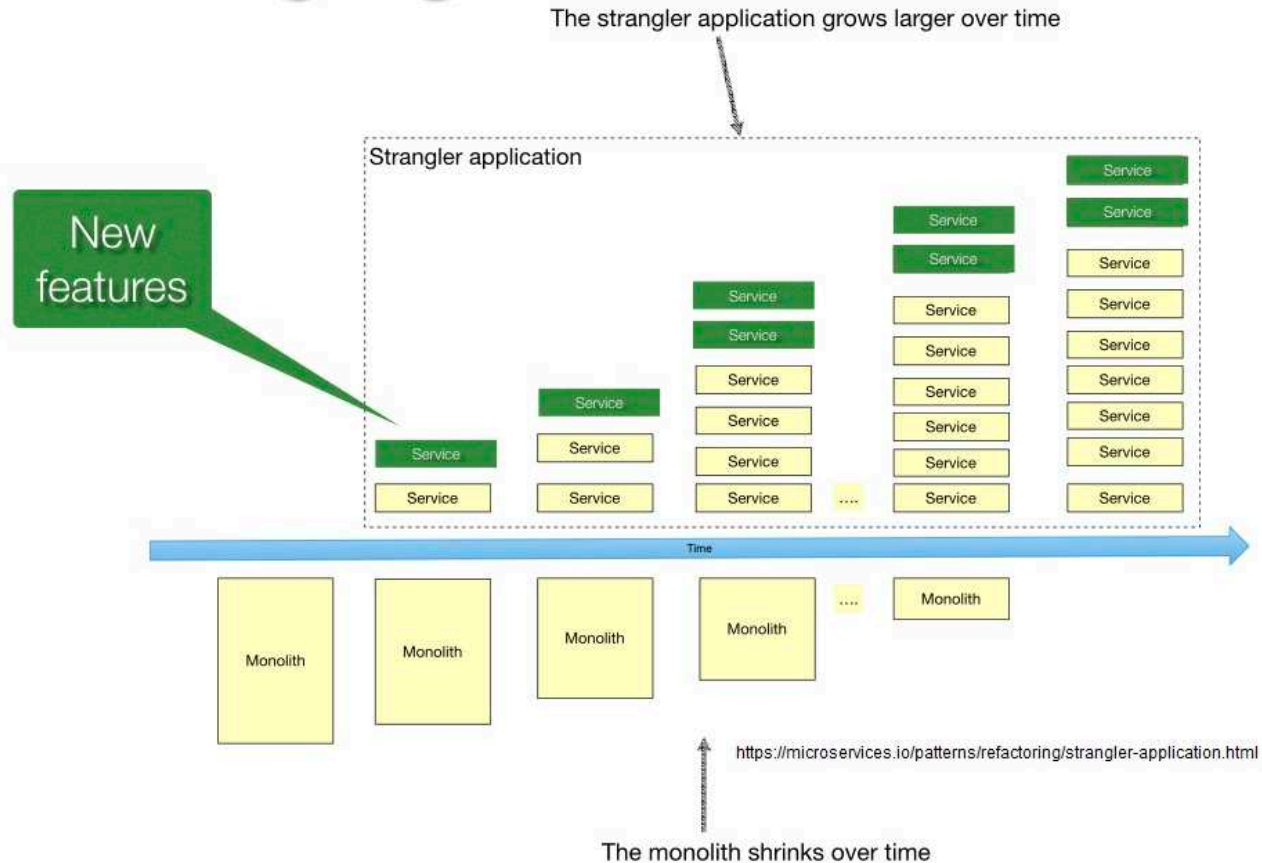
... incomplete

- Data management
- Design and implementation
- Message handling

Catalog:

- https://martinfowler.com/eaaCatalog/
- https://martinfowler.com/books/eaa.html
- https://learn.microsoft.com/pl-pl/azure/architecture/patterns/

Strangling the monolith



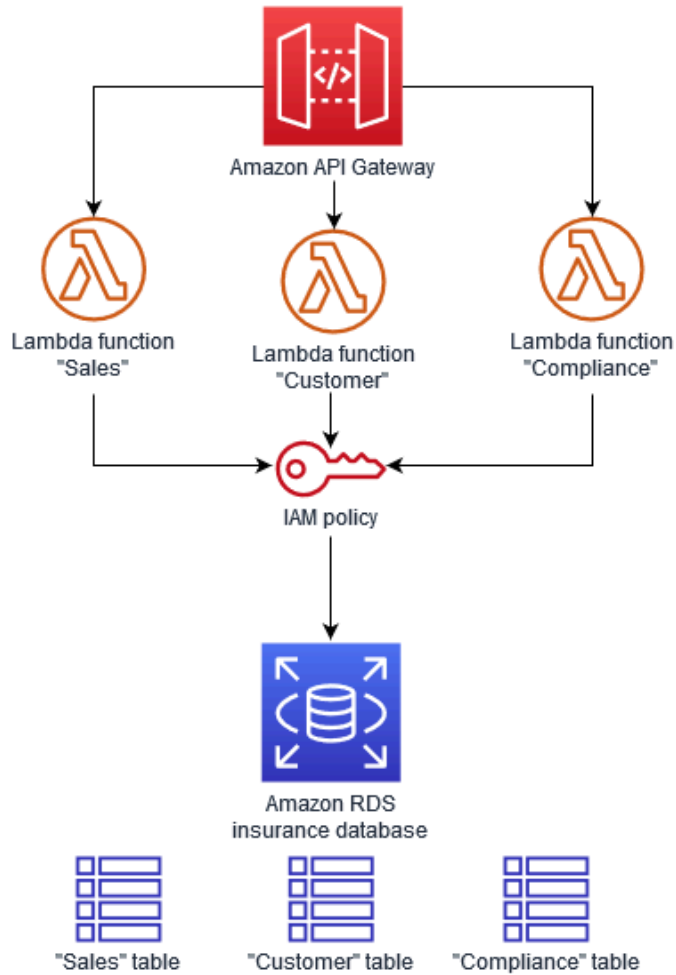
Shared Database

Use a (single) database shared across multiple services.

Each service freely accesses data owned by other services using local ACID transactions.

Disadvantages:

- High coupling between services
- A single database might not meet the storage and access needs of all services



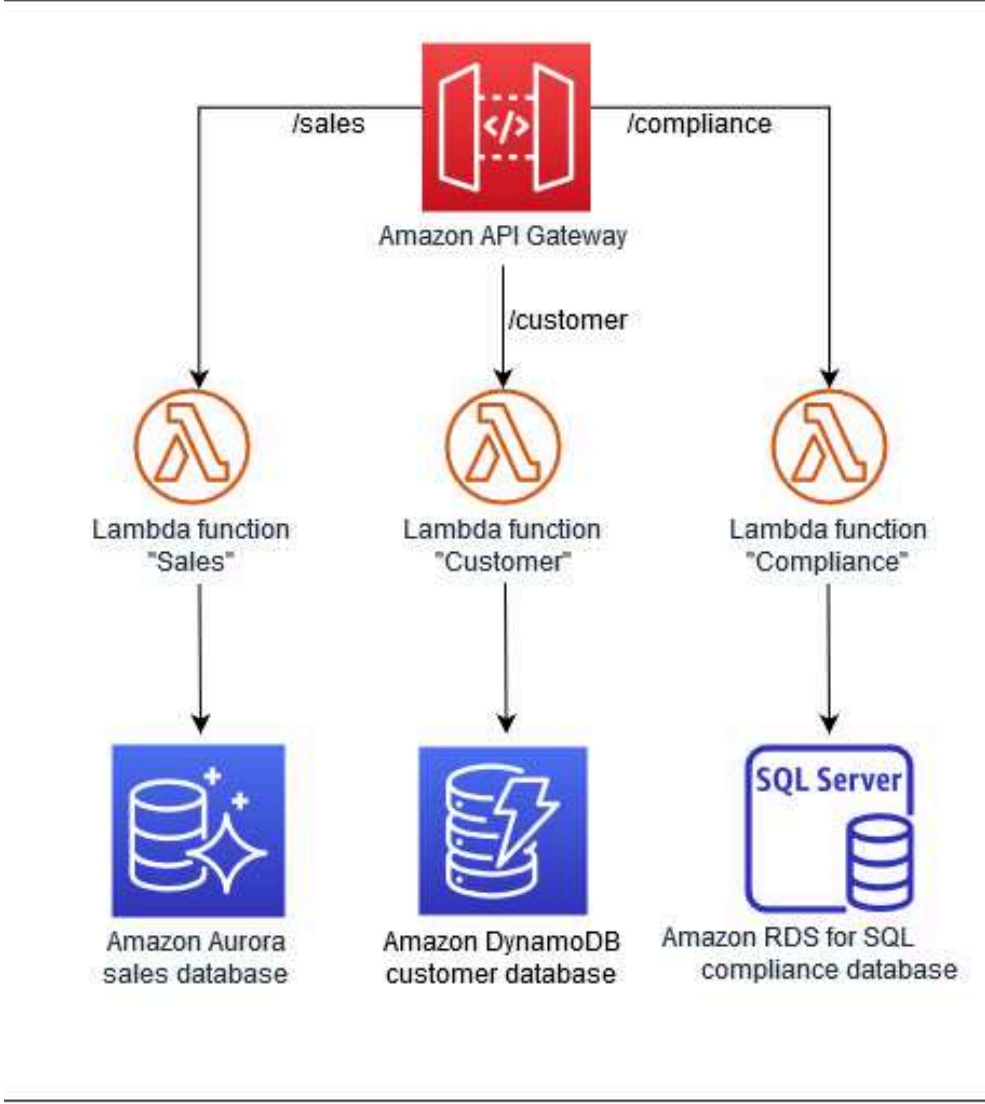
Database per Microservice

Provides many benefits, especially for systems that need to evolve quickly and scale massively.

- Domain data is encapsulated within the service
- The data schema can evolve without directly impacting other services
- Each data store can be scaled independently
- A failure in one service's data store won't directly impact others

Disadvantages:

- Poor data division or slow synchronization



ACID Database

Atomicity, Consistency, Isolation, Durability

Transactions must be atomic, consistent, isolated, and durable (ACID)

- Atomicity – each transaction is either fully completed or not executed at all
- Consistency – the system remains consistent after a transaction
- Isolation – concurrently running transactions don’t usually see each other’s changes
- Durability – the system can restart and provide consistent, intact, up-to-date data

ACID – a set of properties ensuring correct transaction processing in databases

Distributed Transactions

A transaction is, among other things, a set of related tasks that either succeed (commit) or fail (rollback) as a unit.

A distributed transaction affects multiple resources.

To commit a distributed transaction, all participants must guarantee that any data changes will be permanent.

Changes must persist despite system failures or other unforeseen events.

If even one participant cannot guarantee this, the entire transaction fails, and all data changes within it are rolled back.

Eventual Consistency

Eventual consistency is a consistency model used in distributed computing to achieve high availability.

Informally guarantees that if no new updates are made to a data item, eventually all accesses will return the last updated value.

Also known as optimistic replication, it is widely used in distributed systems and originated in early mobile computing designs.

A system that reaches eventual consistency is said to have achieved convergence or replica convergence.

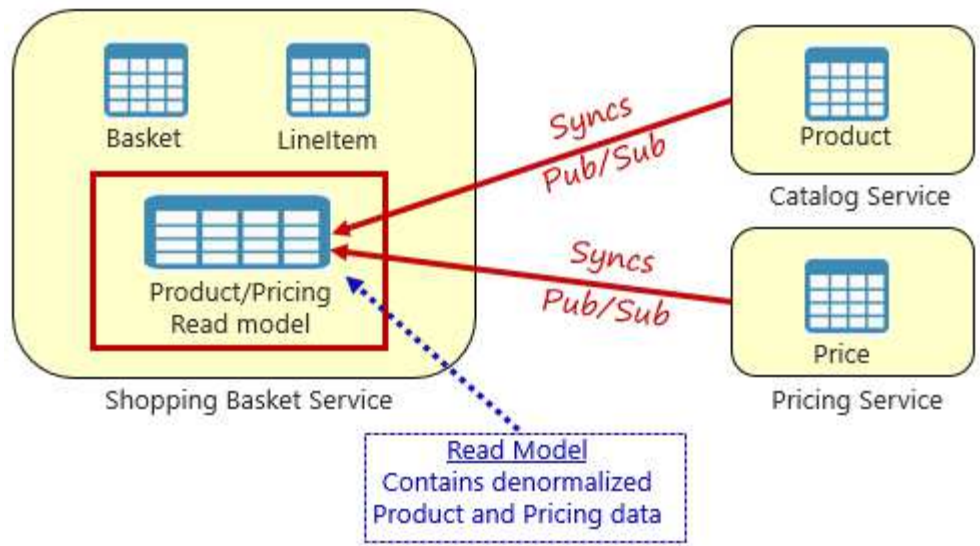
- We don’t have ACID (atomicity, consistency, isolation, durability)
- BASE semantics (Basically Available, Soft state, Eventual consistency)

Pattern: Materialized View

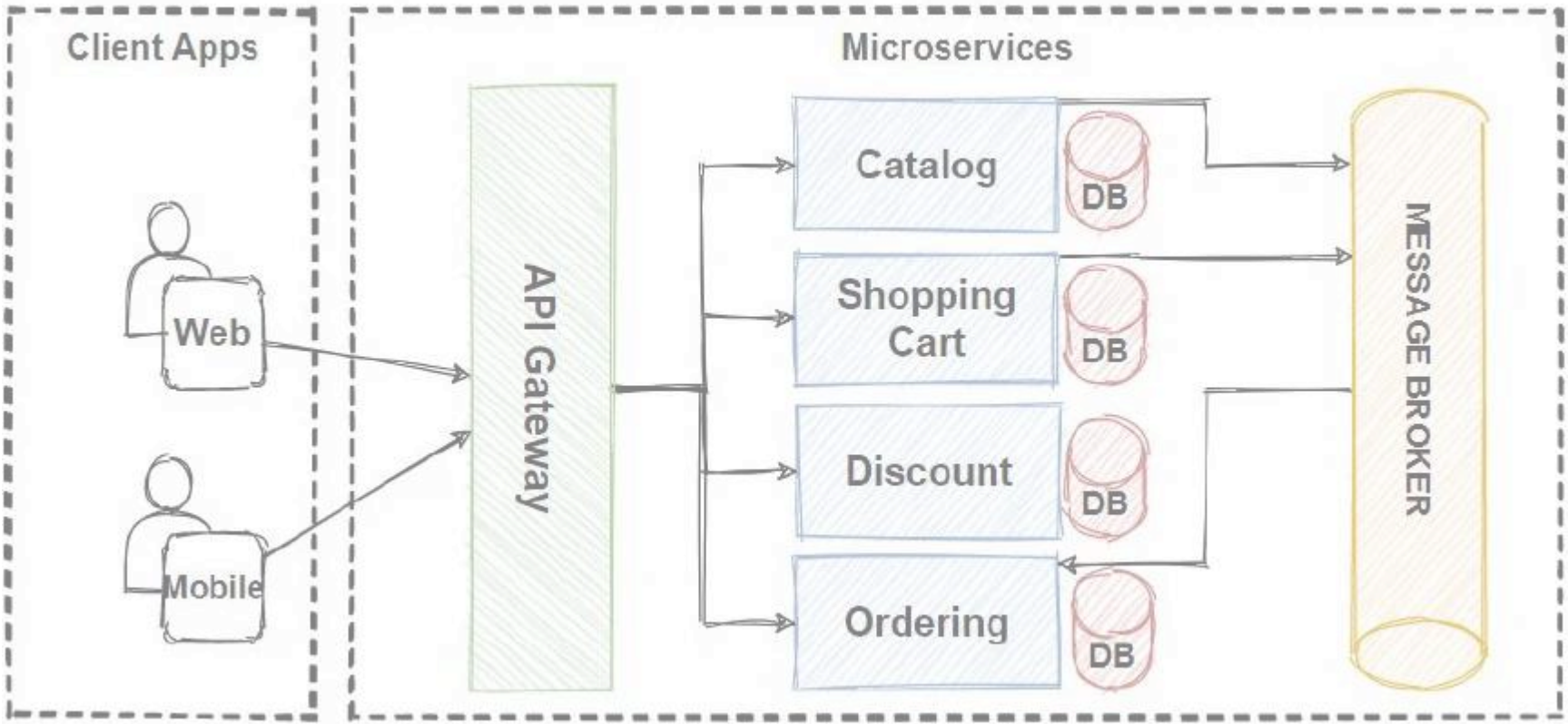
Generate views across several databases when data is not formatted for required operations.

Can help with efficient querying and data extraction, and improve application performance.

- Write in normalized form
- Notify services of changes
- Materialize topic-specific views



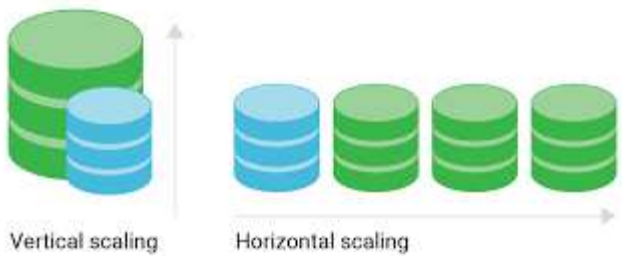
Message Broker



Unhandled or erroneous messages go to the "Dead Letter Channel"

Database Scaling

- Vertical and horizontal scaling
- Read scaling – Replication
- Write (and read) scaling – Sharding

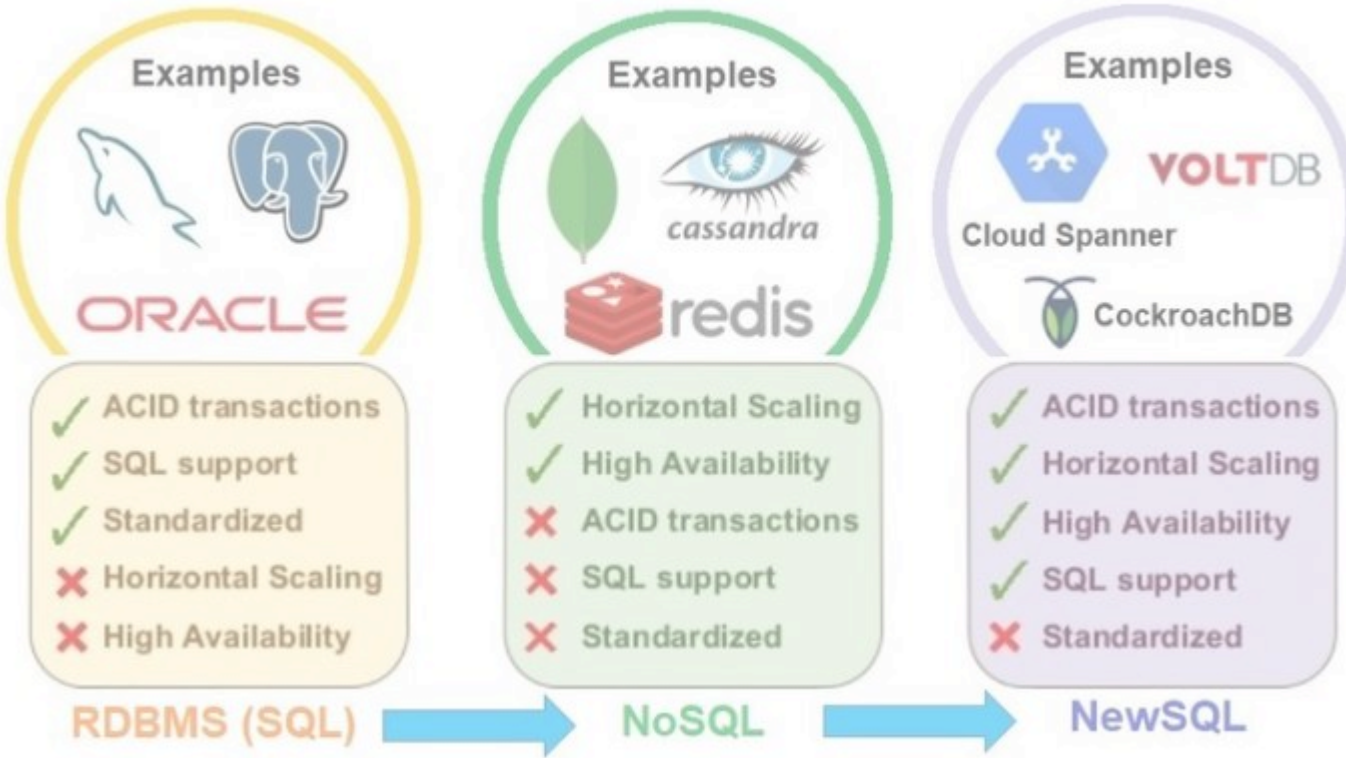


https://learn.microsoft.com/pl-pl/azure/architecture/patterns/sharding

CAP Theorem

Consistency, Availability, Partition Tolerance

SQL, NoSQL, NewSQL



SQL vs NoSQL

SQL Databases

- Tables with fixed rows and columns
- Relationships via foreign keys (JOIN)

Scaling – Vertical

NoSQL Databases

- Document: JSON documents
- Key-Value: key-value pairs
- Wide Column: tables with dynamic rows and columns
- Graph: nodes and edges

Scaling – Horizontal



SQL Patterns

Data Source

- Active Record
- Data Mapper

Object-Relational Behavioral Patterns

- Object-Relational Mapping (ORM)
- Unit of Work
- Identity Map
- Lazy Load
- Query Object
- Repository

Identity Field
Foreign Key Mapping
Association Table Mapping

Migrations

Advantages of NoSQL

NoSQL databases offer many advantages over relational databases.

Flexible data models – NoSQL databases usually have very flexible schemas. This allows easy schema changes as requirements evolve.

Horizontal scaling – Enables horizontal scaling by adding more standard servers as needed.

Fast queries – Data is typically stored in a query-optimized format. Frequently accessed data should be stored together.

Developer-friendly – Some NoSQL databases like MongoDB map their data structures to those of popular programming languages.

Disadvantages of NoSQL

Many NoSQL databases do not support multi-document ACID transactions.

With proper schema design, single-record atomicity is acceptable in many use cases.

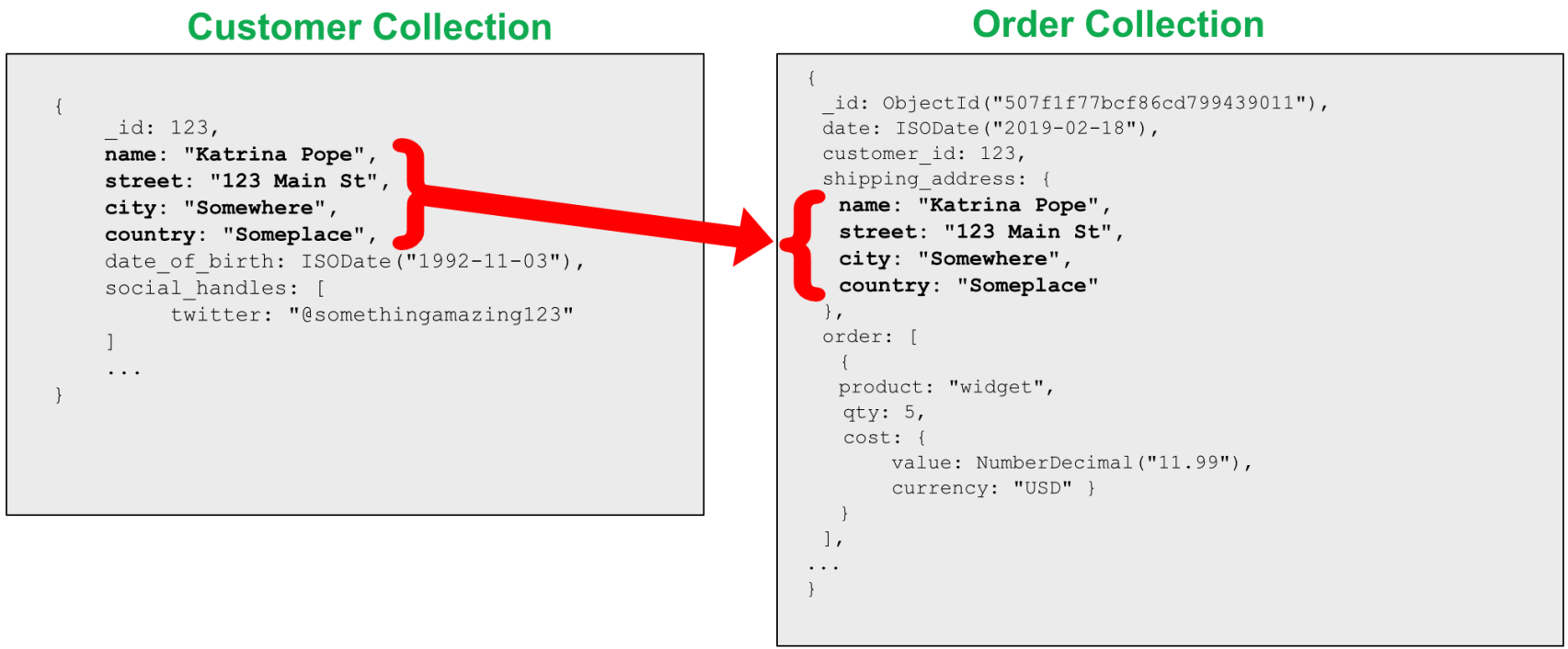
However, many applications still require ACID across multiple records.

NoSQL Patterns

- Polymorphic pattern
- Attribute pattern
- Bucket pattern
- Outlier pattern
- Computed pattern
- Subset pattern

NoSQL – Extended Reference Pattern

Instead of duplicating all client info, copy only the frequently accessed fields.



NoSQL Patterns – Subset

Instead of storing all reviews with the product, split them into two collections.

One collection contains frequently accessed data (e.g., recent reviews), and the other contains less frequently accessed data (e.g., old reviews, product history).

We may duplicate part of the 1-N or N-N relationships used by the most-accessed side of the relation.

```
_id: ObjectId("507f1f77bcf86cd799439011"),
name: "Super Widget",
description: "This is the most useful item in your toolbox.
price: { value: NumberDecimal("119.99"), currency: "USD" },
reviews: [
     {
       review_id: 786,
       review_author: "Kristina",
       review_text: "This is indeed an amazing widget.",
       published_date: ISODate("2019-02-18")
     },
     {
       review_id: 785,
       review_author: "Trina",
       review_text: "Very nice product, slow shipping.",
       published_date: ISODate("2019-02-17")
     },
...
     {
       review_id: 1,
       review_author: "Hans",
       review_text: "Meh, it's okay.",
       published_date: ISODate("2017-12-06")
     }
]
```

NoSQL Patterns – Polymorphism

Selected Service Integration Patterns

Pattern: Scheduler-Agent Supervisor

Coordinate a set of actions across a distributed set of services and remote resources.

Two domain services:

- Scheduler – coordinates the steps
- Supervisor – monitors step status – whether any failed or timed out

This is a variation of the scheduler-agent supervisor pattern.

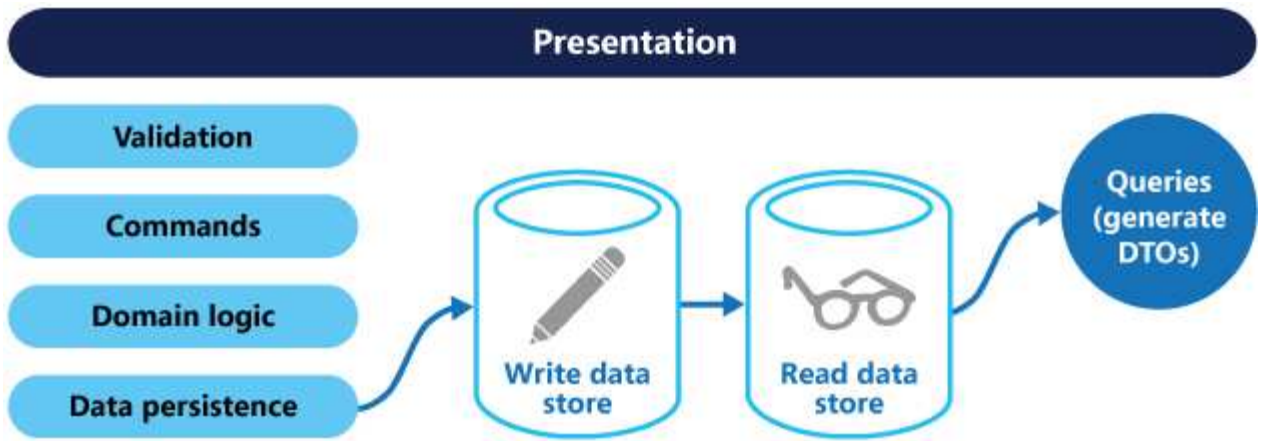
Command and Query Responsibility Segregation

CQRS separates commands from queries – a pattern that divides read and update operations for data stores.

Write and read mechanisms are separated and optimized for their specific use cases.

The pattern introduces delayed data consistency

Data becomes consistent "shortly"



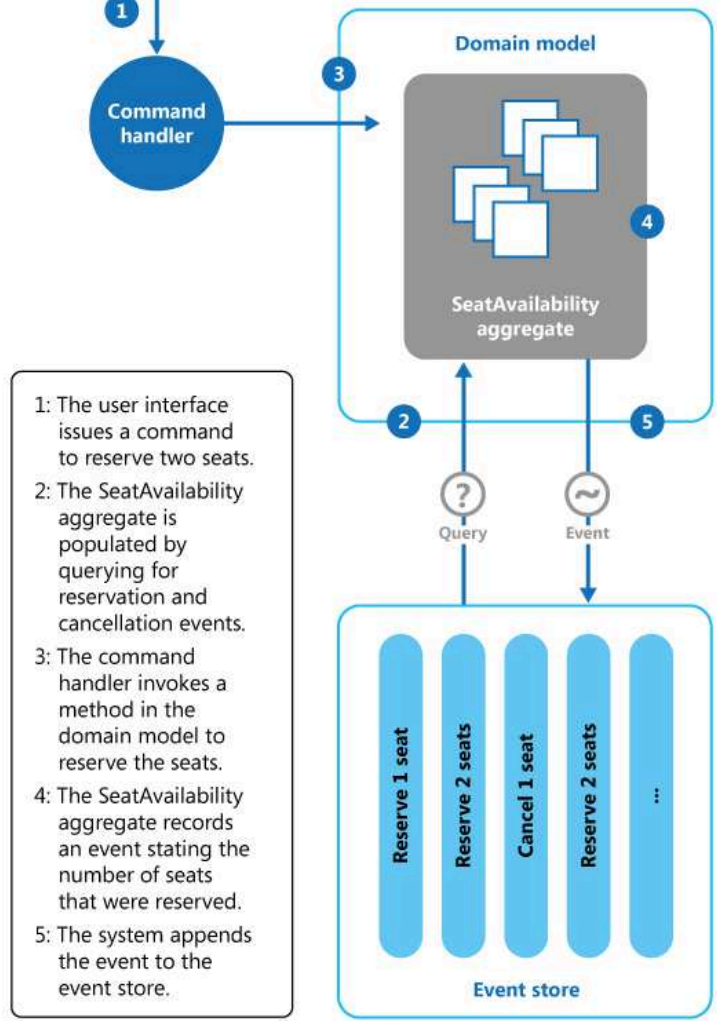
https://learn.microsoft.com/pl-pl/azure/architecture/patterns/cqrs

Event Sourcing Pattern

Instead of storing the current state, record the full sequence of actions performed on the data.

The store acts as a log and can be used to rehydrate domain objects.

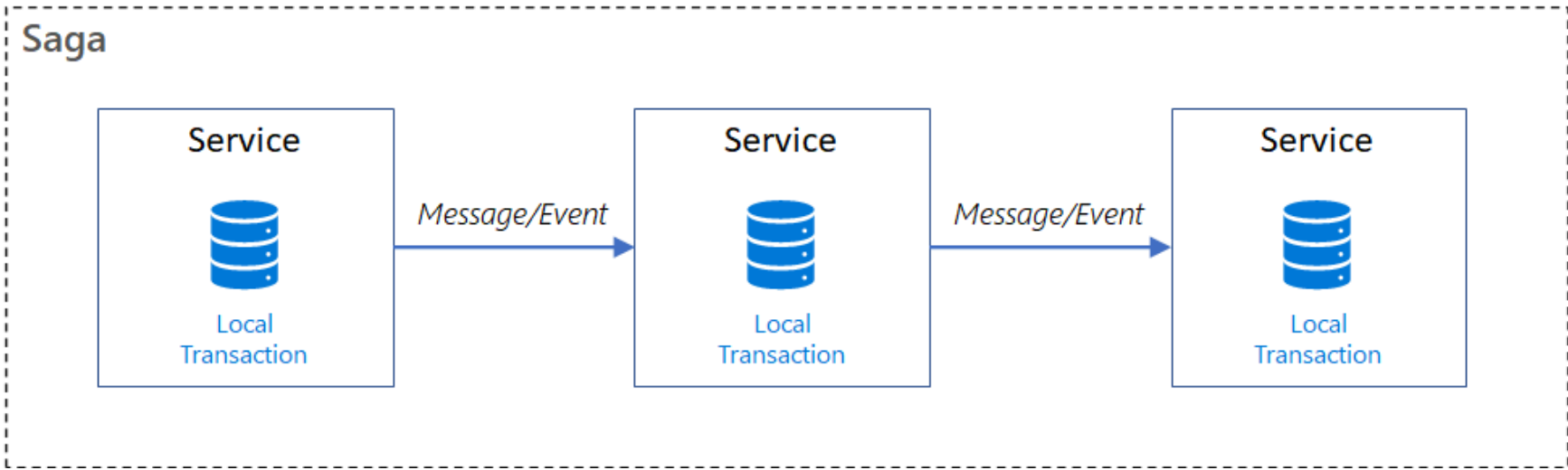
Helps simplify tasks in complex domains by avoiding synchronization between the data model and business domain, while improving performance, scalability, and flexibility.



Distributed Transaction Pattern – Saga

The saga design pattern manages data consistency across microservices in distributed transaction scenarios.

A saga is a sequence of transactions, where each service updates its data and publishes an event to trigger the next step.

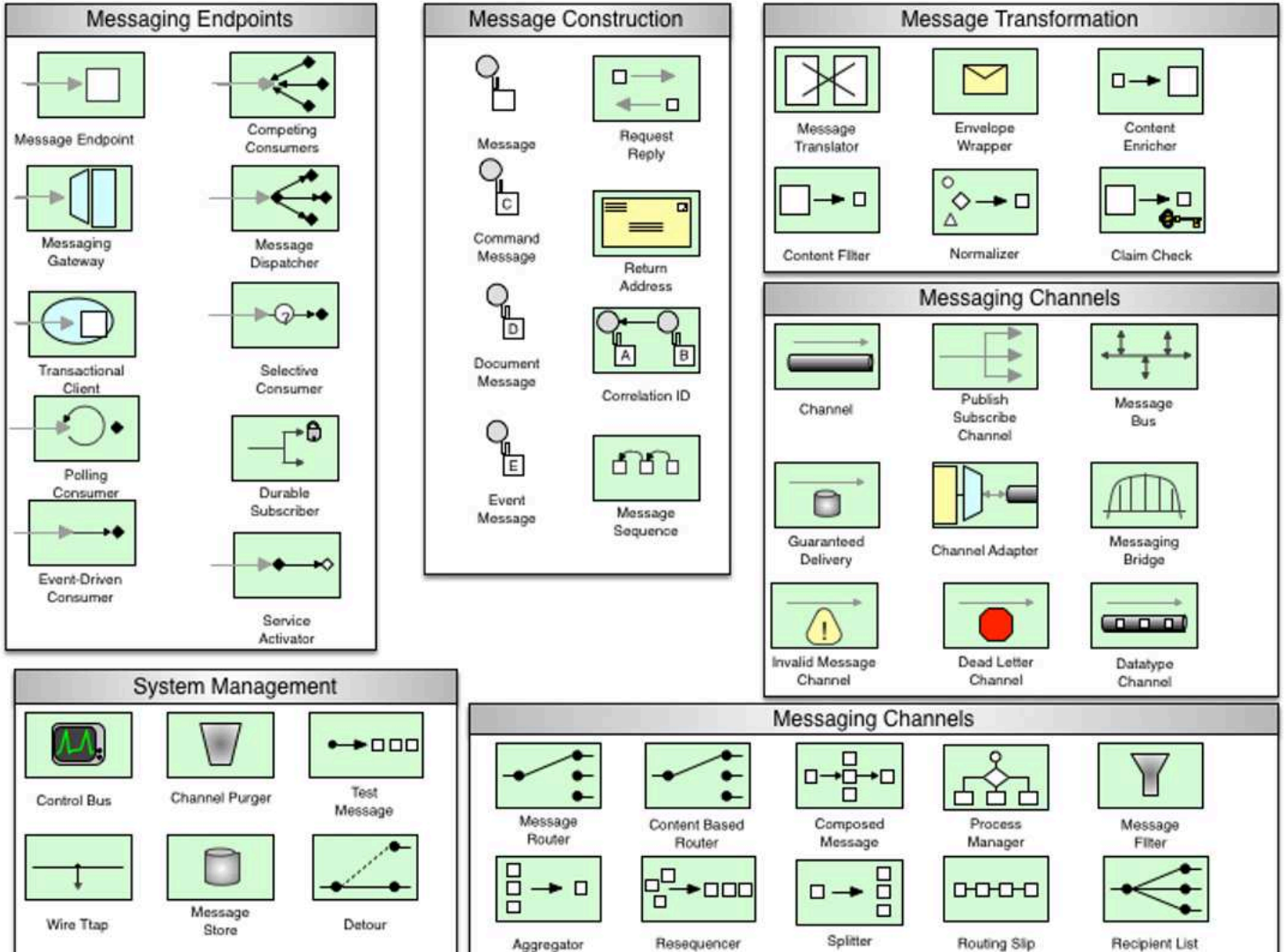


If a step fails, the saga executes **compensating transactions** to undo the effects of previous steps.

Enterprise Integration Patterns

Enterprise Integration Patterns help design message-based systems by defining common solutions for routing, transformation, and system coordination.

These patterns are foundational for building message-oriented and distributed architectures.


Messaging Endpoints
Message Endpoint
Competing Consumers
Messaging Gateway
Message Dispatcher
Transactional Client
Selective Consumer
Polling Consumer
Durable Subscriber
Event-Driven Consumer
Service Activator
Message Construction
Message
Request Reply
Command Message
Return Address
Document Message
Correlation ID
Event Message
Message Sequence
Message Transformation
Message Translator
Envelope Wrapper
Content Enricher
Content Filter
Normalizer
Claim Check
Messaging Channels
Channel
Publish Subscribe Channel
Message Bus
Guaranteed Delivery
Channel Adapter
Messaging Bridge
Invalid Message Channel
Dead Letter Channel
Datatype Channel
System Management
Control Bus
Channel Purger
Test Message
Wire Ttap
Message Store
Detour
Messaging Channels
Message Router
Content Based Router
Composed Message
Process Manager
Message Filter
Aggregator
Resequencer
Splitter
Routing Slip
Recipient List

Message Channel

A conduit through which messages are sent between systems or components.

Use Case:

An e-commerce platform sends OrderPlaced messages to a fulfillment service via an "orders" message channel.

Tools: RabbitMQ, Kafka, NATS

Message

The fundamental unit of data exchanged between components, often with a header and a body.

Use Case:

A message containing customer details and purchase items is sent to the CRM system.

Tools: JSON, Protobuf, Avro, AMQP message formats

Message Router

Routes messages to different destinations based on message content or rules.

Use Case:

A payment service routes transactions with amount > $1000 to a fraud detection service.

Tools: Apache Camel, Spring Integration, NServiceBus

Message Filter

Removes messages that do not meet certain criteria before further processing.

Use Case:

Filter out non-priority support tickets before passing them to the escalation system.

Tools: Apache Camel filter(), Spring Integration Filter

Message Translator

Converts message formats so systems with different data models can communicate.

Use Case:

Translate an internal JSON order message to an external SOAP request format for a legacy ERP.

Tools: Apache Camel, Node.js middleware, XSLT

Content Enricher

Adds missing data to a message by calling external services or databases.

Use Case:

Enrich a shipment message with warehouse location data before forwarding to the logistics system.

Tools: Spring Integration Transformer, Apache Camel enrich()

Content Filter

Removes unnecessary or sensitive parts of a message before forwarding.

Use Case:

Strip customer PII before sending transaction data to analytics service.

Tools: Middleware, masking libraries, message mappers

Message Broker

Centralized system that routes, stores, and forwards messages between services.

Use Case:

A logistics system relies on a broker to publish and subscribe to shipping and tracking events.

Tools: RabbitMQ, Apache Kafka, Azure Service Bus

Splitter

Breaks a composite message into individual messages for separate processing.

Use Case:

Split a bulk product import into individual product messages for validation.

Tools: Apache Camel split(), Spring Integration Splitter

Aggregator

Combines related messages into a single message after receiving all parts.

Use Case:

Collect multiple shipment confirmations and aggregate them into one delivery update for the user.

Tools: Apache Camel aggregator(), Kafka Streams

Claim Check

Stores part of the message temporarily and replaces it with a reference (claim check).

Use Case:

Store large image data in blob storage and pass a URL reference through the message.

Tools: External storage (S3, Redis) + message metadata

Dead Letter Channel

Captures messages that could not be processed successfully for later inspection.

Use Case:

A failed payment event is sent to the dead letter queue for manual review.

Tools: RabbitMQ DLX, Kafka dead-letter topic, AWS SQS DLQ

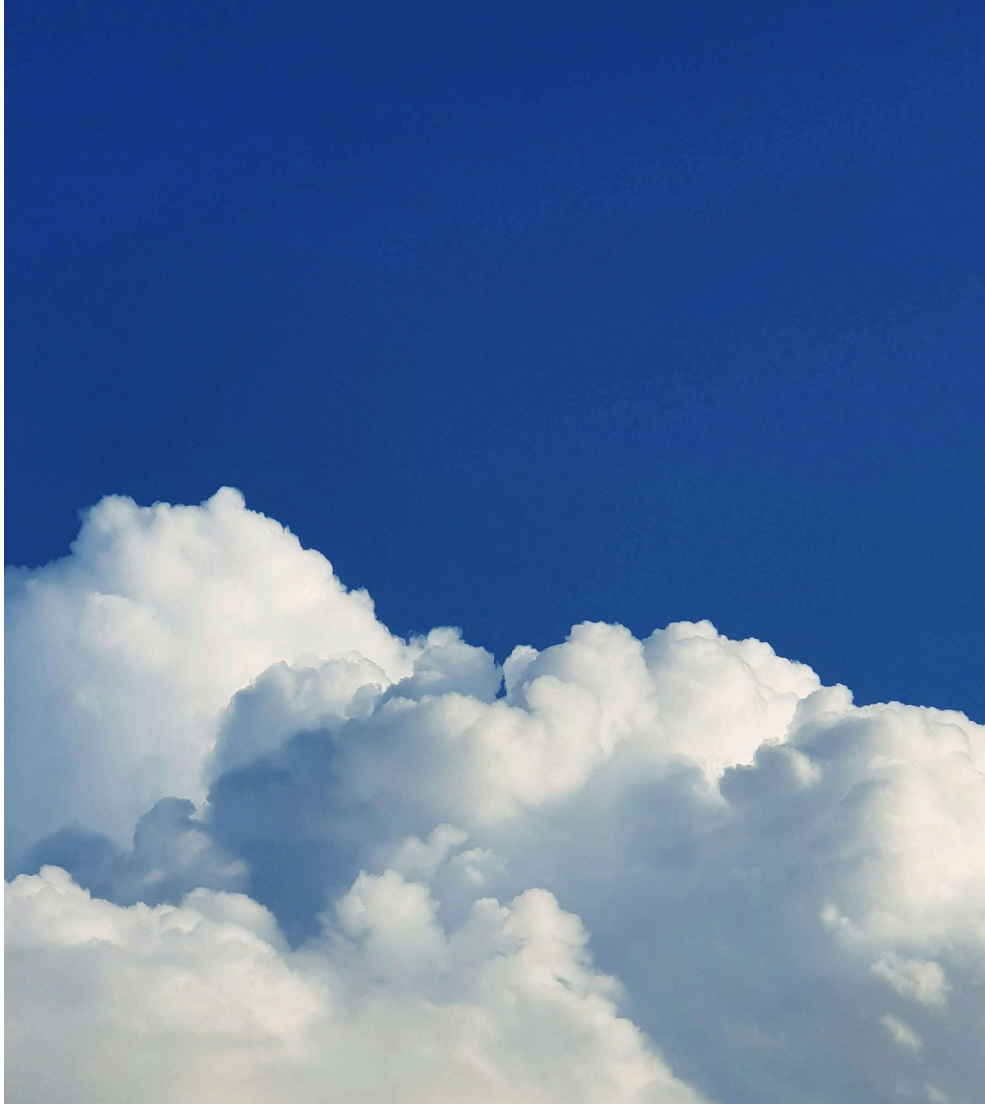
Cloud

Concepts, patterns and technologies

False Assumptions

about the cloud

- The network is reliable;
- Latency is zero;
- Bandwidth is infinite;
- The network is secure;
- Topology doesn’t change;
- There is one administrator;
- Transport cost is zero;
- The network is homogeneous.



Distributed System

— a set of independent computers that appear to users as a single, logically cohesive system.

Resource sharing — a distributed system can share hardware, software, or data.

Concurrent processing — many computers can process the same function simultaneously.

Scalability — computing and processing performance can be scaled as needed by adding more computers.

Fault detection — failures can be more easily detected.

Transparency — a node can access and communicate with other nodes in the system.

- partitioning and scale
- location
- routing
- relocation
- replication
- concurrency
- failure
- persistence

What is distributed tracing?

Distributed tracing is a method used to profile or monitor the outcome of a request as it travels through a distributed system.

Monitoring a distributed system can be difficult because each node has its own stream of logs and metrics.

To get an accurate view of the distributed system, these separate node metrics must be aggregated into a holistic picture.

Requests to distributed systems typically do not access the entire set of nodes but a portion of that set or a path through nodes.

Distributed tracing reveals the most commonly used paths in the system and allows teams to analyze and monitor those paths.

Benefits of Distributed Systems

Distributed systems often help improve the reliability and performance of a system.

Improved reliability is achieved by eliminating central points of failure and bottlenecks.

Nodes in a distributed system provide redundancy so that if one node fails, others are ready to take over its function.

Performance improves because nodes can be easily scaled horizontally and vertically.

If the system experiences high load, more nodes can be added to handle it.

To support heavier loads, a single node’s performance can also be increased.

Drawbacks and Risks of Distributed Systems

As a system becomes too complex, its maintenance becomes demanding.

With increasing system complexity, teams may find it difficult to effectively organize, manage, and improve the system.

It can be challenging to understand how different components are connected or who owns a specific software component.

This complicates making changes in a way that maximizes operational efficiency and avoids negative effects on dependencies and customers.

When the system includes multiple repositories, specialized tools may be required to organize and manage distributed system code.

Distributed System Architectures

Client-server — This architecture has two main responsibilities. The client handles user interface presentation and connects over a network to a server. The server handles business logic and state management.

Multi-tier architecture — The server is divided into more granular nodes that allow additional responsibilities to be extracted from the server.

Peer-to-peer architecture — Each node contains a complete instance of the application. There is no division into presentation and data processing nodes. Peer-to-peer systems are extremely redundant.

Service-oriented architecture — SOA is the precursor to microservices. The main difference between SOA and microservices lies in the scope of nodes — microservices include business logic for a specific set of functions, e.g., payment processing.

Elastic Systems

- Easy configuration and reconfiguration of topology
- Separation of scaling policy from mechanism

Redundancy — if a node fails, its function can be taken over by others.

Each node can be scaled horizontally and vertically to increase performance.

If the system experiences high load, more nodes can be added to handle it.

Replication

- increase availability
- increase durability
- load balancing
- caching
- data consistency

Cloud Computing

Resource elasticity, faster innovation, and efficient use of economies of scale.

Cloud computing is the delivery of computing resources — including storage, compute power, databases, networking, analytics, AI, and software — over the Internet via the cloud.

Outsourcing this area allows companies to access needed computing resources at any time without buying and maintaining on-premise IT infrastructure.

Cloud - Characteristics

On-demand self-service — APIs allow users to request new resources and scale existing ones as needed. This enables efficient infrastructure provisioning automation using "infrastructure as code" tools like Terraform and Ansible.

Broad network access — enabled by globally distributed physical infrastructure that allows for strategic provisioning of localized services.

Resource pooling — Resources are dynamically divided and allocated on demand. The physical hardware of a cloud host is shared and optimized for maximum use across tenants.

Rapid elasticity — Dynamic scaling up and down enables automatic compute resource scaling depending on user needs. This can apply to a single machine or multiple devices scaled by the application.

Measured service — Detailed usage metrics help manage costs. For example, AWS provides hourly or daily usage stats per service category.

Cloud providers typically use a pay-as-you-go model — measured and delivered so customers only pay for the compute resources they actually use.

Cloud Advantages

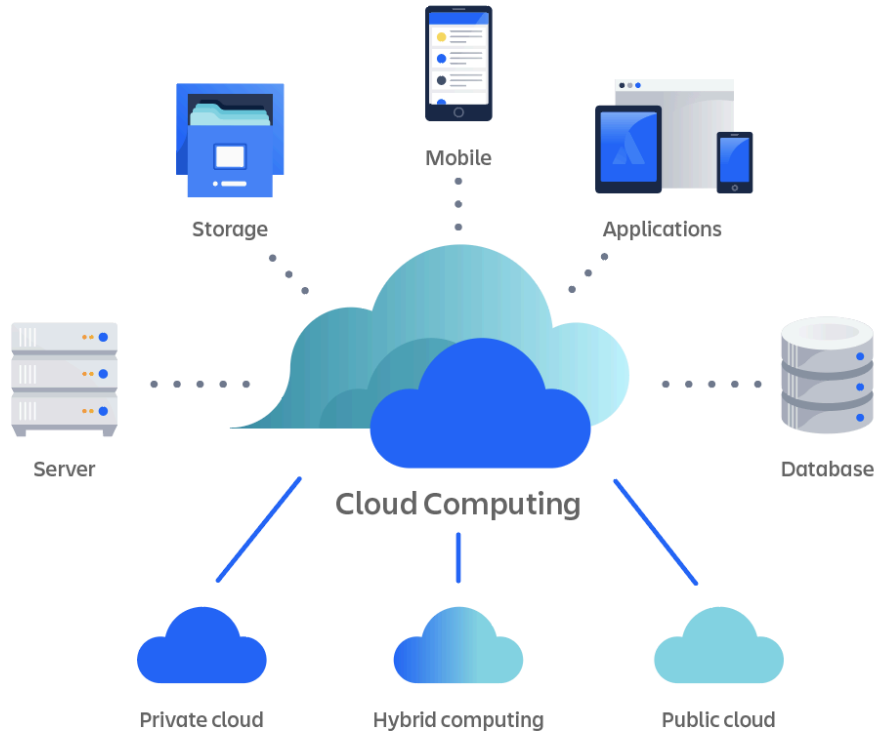
Cost reduction — Teams using cloud resources don't need to buy their own hardware. Providers also maximize and optimize device usage.

Increased scalability — The inherent elasticity of cloud solutions allows on-demand resource scaling.

Better performance — Cloud computing provides access to cutting-edge compute solutions.

Improved security — Hosting in a private cloud means isolated infrastructure protected by a firewall for enhanced security.

Comprehensive monitoring and incident management — Cloud deployments allow integration of tools across the entire process pipeline, aiding in full visibility. Full monitoring is essential for DevOps practices.


Mobile
Storage
Applications
Server
Database
Cloud Computing
Private cloud
Hybrid computing
Public cloud

Cloud Deployment Models

Public cloud — provides computing resources (servers, storage, apps, etc.) over the Internet from providers like AWS or Microsoft Azure.

Private cloud — computing resources dedicated to a single organization, either physically located in the company's own data center or hosted by a provider.

Hybrid cloud — combines public and private clouds (e.g., IBM Hybrid Cloud with Red Hat software)

Cloud as a Service

- VPS - Virtual Private Server
- SaaS - Software as a Service
- PaaS - Platform as a Service
- IaaS - Infrastructure as a Service
- FaaS - Function as a Service (Serverless)

Containers

Containers and microservices are not the same.

Benefits of containerization:

- **Portability** — A container image is a self-contained package that runs without installing libraries or other dependencies. Containers can be quickly launched or stopped, allowing for load increases or node recovery.
- **Density** — Containers are lightweight compared to virtual machines as they share the OS. You can pack many containers on a single node, useful for apps composed of many small services.
- **Resource isolation** — You can limit memory and CPU available to a container, ensuring runaway processes don’t exhaust host resources.

What is Docker?

A commercial containerization platform and runtime environment that helps developers build, deploy, and run containers.

Uses a client-server architecture with simple commands and automation via a single API.

Docker also provides a toolset widely used to package applications into immutable container images via a Dockerfile and commands to build images using the Docker daemon.

Coordinating and scheduling container workloads across multiple servers/clusters requires orchestration.

Kubernetes, Docker Swarm, Mesos, HashiCorp Nomad, etc. — these allow organizations to manage large numbers of containers and users.

What is Kubernetes?

(often called K8s) is a popular open-source platform for orchestrating container runtimes in a networked resource cluster.

Kubernetes groups containers into Pods that can be managed as a unit on a single machine to reduce network load and increase resource efficiency.

A Pod might include an app server, Redis cache, and SQL database. Docker containers run one process per container.

Kubernetes is not a full Platform as a Service (PaaS), and when building or managing Kubernetes clusters, many aspects must be considered. Public platforms simplify working with Kubernetes.

Kubernetes Benefits

Often called the “Linux of the cloud,” it’s the most popular container orchestration platform for good reason.

Automated operations — advanced API and CLI tool called `kubectl`. Kubernetes controller patterns ensure apps/containers run exactly as intended.

Infrastructure abstraction — manages available resources so developers can focus on app code, not the underlying compute, networking, or storage infrastructure.

Service monitoring — watches the live environment and compares it to the desired state. Automatically performs health checks and restarts failed or stopped containers. Only exposes services when they’re up and ready.

Kubernetes separates description from execution — we describe WHAT the system should look like, and it makes the necessary changes.

Serverless Architecture

Allows developers to focus on code, not servers.

In this architecture, you pay for infrastructure availability on demand — only when you or your customers are using server resources.

In a serverless model, you don’t worry about infrastructure maintenance, updates, licenses, scaling, performance, configurations, backups, or admin teams.

Massive scalability is also a risk
— huge...

Serverless Architecture

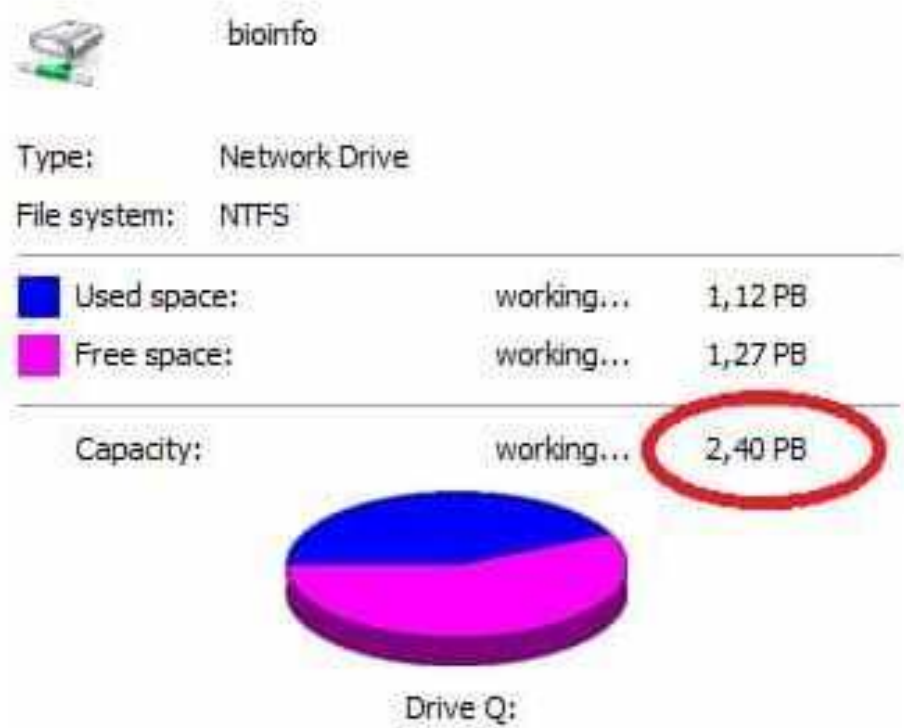
- Simplified backend code – we can create simple functions that independently fulfill specific tasks
- Simplified scalability – we don’t have to worry about scaling the code
- Lower costs – we pay less than with cloud setups involving server allocation, where money is often spent on unused resources or idle CPU time

Big Data Architecture

is designed to handle the ingestion, processing, and analysis of data that is too large or too complex for traditional database systems.

The threshold that organizations must reach to enter the big data realm varies depending on users' capabilities and their tools.

In some cases, it may mean hundreds of gigabytes of data, in others, hundreds of terabytes.



System Integration

Legacy system integration – involves connecting modern applications with existing or outdated systems

Enterprise application integration – unifies various subsystems within a business environment.

Third-party system integration – is often a popular choice among companies looking to enhance the functionality of their technology without investing in a completely new platform.

Business-to-business (B2B) integration – focuses on automating transactions and exchanging documents between multiple companies. The goal is to improve collaboration and trade with partners, customers, and suppliers.

Benefits of System Integration

Productivity – fully integrated systems allow for more consistent control over daily processes, increasing the efficiency of business workflows. Companies can accomplish more in less time, and employees have access to all necessary information in one environment.

Data accuracy – system integration helps create a single source of truth for all data. This ensures that information shared across the company is consistent, enabling more accurate business data discovery.

Faster decision-making – since information isn't scattered in isolated storage environments, companies can perform analysis and diagnostics more quickly and effectively. A holistic view of all business data enables better decision-making.

Cost-effectiveness – system integration is often a cheaper innovation option for companies than replacing disconnected parts with new technology. It also reduces the need for additional specialized support when tracking data.

Blockchain

is an example of distributed ledger technology architecture.

One of the key goals of blockchain is to reach consensus among its peer-to-peer nodes, meaning that all nodes agree on the current state of the network.

As a result, blockchain nodes maintain identical copies of the data ledger – commonly referred to as the ledger – and using rules defined in the protocol, agree on changes made to it.

Transactions are digitally signed using public key cryptography with elliptic curve cryptography (ECDSA) and are public, although users remain anonymous.

Blockchain

Public Key

- like a lock and key
- like mixing paints

Ledger

- public: Bitcoin, Ethereum, others...
- private: IBM Hyperledger Fabric

Blockchain

- Contracts (Ethereum)
- Tokens (ERC-20)
- Autonomous organizations

Exercises:

- Example of blockchain use
- Example of contract use
- Example of token use

Thank you!

https://www.linkedin.com/in/mateuszkulesza/